BWidget

Reference manual pages

2026-10-27

Contents

ArrowButton **3**
BWidget **6**
Button **8**
ButtonBox **11**
ComboBox **14**
Dialog **18**
DragSite **22**
DropSite **23**
DynamicHelp **26**
Entry **29**
Label **33**
LabelEntry **37**
LabelFrame **39**
ListBox **40**
MainFrame **47**
MessageDlg **51**
NoteBook **53**
PagesManager **57**
PanedWindow **59**
PanelFrame **61**
PasswdDlg **63**
ProgressBar **65**
ProgressDlg **67**
ScrollView **68**
ScrollableFrame **70**
ScrolledWindow **72**
SelectColor **74**
SelectFont **75**
Separator **77**
SpinBox **78**
StatusBar **81**
TitleFrame **82**
Tree **84**
Widget **93**
contents **98**
index **99**
navtree **100**
options **101**
License **110**

ArrowButton

Keywords: ArrowButton

NAME

ArrowButton - Button widget with an arrow shape.

CREATION

ArrowButton *pathName* ?*option value...*?

STANDARD OPTIONS

-activebackground	
-activeforeground	
-background or -bg	-borderwidth or -bd
-disabledforeground	
-foreground or -fg	
-highlightbackground	
-highlightcolor	
-highlightthickness	
-relief	
-repeatdelay	
-repeatinterval	
-takefocus	
-troughcolor	

WIDGET-SPECIFIC OPTIONS

-armcommand
-arrowbd
-arrowrelief
-clean
-command
-dir
-disarmcommand
-height

-helptext
-helptype
-helpvar
-ipadx
-ipady
-state
-type
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

DESCRIPTION

ArrowButton can be of two types following **type** option: for **button** type, it is standard button with an arrow drawn on it; for **arrow** type, it is an arrow like scrollbar's arrow.

WIDGET-SPECIFIC OPTIONS

-armcommand
Specifies a Tcl command to associate with the ArrowButton when mouse button 1 is pressed over the ArrowButton. When **repeatdelay** or **repeatinterval** option is positive integer, this command is repeatedly called if mouse pointer is over the button and until mouse button 1 is released.

-arrowbd
When ArrowButton **type** is *arrow*, specifies the border width of the arrow. Must be 1 or 2.

-arrowrelief
When ArrowButton **type** is *arrow*, specifies the relief of the arrow. Must be **raised** or **sunken**.

-clean
Specifies a level of quality, between 0 and 2, for the arrow. If 0, the arrow is drawn with its maximum width and height. If 1, the base of arrow is arranged to be odd to have same edges. If 2, the base of arrow is arranged to be odd and the orthogonal to be (base+1)/2 to have 'straight' diagonal for edges.

-command
Specifies a Tcl command to associate with the ArrowButton. This command is typically invoked when mouse button 1 is released over the ArrowButton window.

-dir
Specifies the direction of the arrow: **top**, **bottom**, **left** or **right**.

-disarmcommand
Specifies a Tcl command to associate with the ArrowButton when mouse button 1 is released. This command is called even if pointer is not over the ArrowButton, and always before the command specified by **command** option. It is typically used in conjuntion with **armcommand**, **repeatdelay** and **repeatinterval**.

-height
Specifies a desired height for the ArrowButton. The value is in screen units.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-ipadx
Specifies a minimun pad between the ArrowButton border and the right and left side of the arrow. The value is in screen units.

-ipady
Specifies a minimun pad between the ArrowButton border and the top and bottom side of the arrow. The value is in screen units.

-state
Specifies one of three states for the ArrowButton: **normal**, **active**, or **disabled**.
If ArrowButton type is *button*:
In normal state the ArrowButton is displayed using the **foreground** and **background** options. The active state is typically used when the pointer is over the ArrowButton. In active state the ArrowButton is displayed using the **activeforeground** and **activebackground** options. In disabled state the **disabledforeground** and **background** options determine how the ArrowButton is displayed.
If ArrowButton type is *arrow*:
Only colors of arrow change. The background of ArrowButton is always displayed using **troughcolor** option. In normal state the ArrowButton is displayed using the **background** option. The active state is typically used when the pointer is over the ArrowButton. In active state the ArrowButton is displayed using the **activebackground** option. In disabled state the ArrowButton is displayed with a dark stipple.

Disabled state means that the ArrowButton should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses.

-type
Determines the type of the ArrowButton: **button** for standard button look, or **arrow** scrollbar's arrow look.

-width

Specifies a desired width for the ArrowButton. The value is in screen units.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* invoke**
If ArrowButton **state** is not disabled, this invoke the commands of the button. ArrowButton is redisplayed with active color and sunken relief, and **armcommand** is called. Then ArrowButton is redisplayed with normal color and its defined relief, and **disarmcommand** then **command** are called.

invoke is called when ArrowButton has input focus and user press the space bar.

BWidget

Keywords: BWidget

Under construction ...

NAME
BWidget - Description text

COMMAND
BWidget::XLFDfont *cmd* ?*arg...*?

BWidget::assert *exp* ?*msg*?

BWidget::badOptionString *type value list*

BWidget::bindMouseWheel *widget*

BWidget::classes *class*

BWidget::clonename *menu*

BWidget::focus *option path*

BWidget::get3dcolor *path bgcolor*

BWidget::getname *name*

BWidget::grab *option path*

BWidget::inuse *class*

BWidget::library *class* ?*class ...*?

BWidget::lreorder *list neworder*

BWidget::parsetext *text*

BWidget::place *path w h* ?*arg...*?

BWidget::write *filename* ?*mode*?

BWidget::wrongNumArgsString *string*

DESCRIPTION

Description text

COMMAND

BWidget::XLFDfont *cmd* ?*arg...*?
Description text

BWidget::assert *exp* ?*msg*?
Description text

BWidget::badOptionString *type value list*
Return a proper error string for a *value* of *type* that doesn't match *list*.

BWidget::bindMouseWheel *widget*
Bind the given *widget* with the standard mouse wheel bindings.

BWidget::classes *class*
Returns a list of classes needed by the given *class*.

BWidget::clonename *menu*
Description text

BWidget::focus *option path*
Description text

BWidget::get3dcolor *path bgcolor*
Description text

BWidget::getname *name*
Description text

BWidget::grab *option path*
Description text

BWidget::inuse *class*
Returns true or false if the given *class* is being used by the current running program.

BWidget::library *class ?class ...?*
Returns a string of code that contains all the libraries needed to use the widgets given by *class*. Each class's code and the code of its dependent classes is appended to the string and returned. This is mostly useful for saving BWidgets to another project.

BWidget::lreorder *list neworder*
Description text

BWidget::parsetext *text*
Description text

BWidget::place *path w h ?arg...?*
Used to position and resize the widget specified by *path*. *w* and *h* are used to specify the requested width and height of the *path* widget for use by **wm geometry** (set to 0 to use current values). The placement of the widget relative to other widgets or the screen is controlled by additional arguments:

at *x y*
Place the widget specified by the *path* argument at screen position x,y. See **wm geometry** for information about window placement values.

center*?widget?*
Place the *path* widget centered against *widget* or against the root widget if *widget* is not given.

right*?widget?*
Place the *path* widget either left or right of the reference widget (or the root widget if *widget* is not specified). If the reference widget's position is such that the newly placed window might be obscured then the opposite side will be tried.

below*?widget?*
As for left/right above, this option causes the widget to be placed either above or below the reference widget with the opposite placement being attempted if the newly placed widget will not be visible.

BWidget::write *filename ?mode?*
Writes the currently used set of BWidget class code to the given *filename*. All the code necessary to run the BWidgets currently in use is written to the file. This is mostly useful for saving BWidget code to another project as a single file instead of the entire BWidget package.

BWidget::wrongNumArgsString *string*
Returns a standard error string for the wrong number of arguments. *string* is appended to the standard string.

Button

Keywords: Button

NAME

Button - Button widget with enhanced options

CREATION

Button *pathName* ?*option value...*?

STANDARD OPTIONS

-activebackground	-activeforeground
-anchor	-background or -bg

WIDGET-SPECIFIC OPTIONS

-armcommand	-command
-default	-disarmcommand
-height	-helptext
-helptype	-helpvar
-name	-relief
-state	-underline
-width	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

DESCRIPTION

Button widget extends the Tk button with new options. DynamicHelp options, a new relief style, callback for **arm**/**disarm**, and **repeatdelay**/**repeatinterval** options has been added.

WIDGET-SPECIFIC OPTIONS

-armcommand

Specifies a Tcl command to associate with the Button when mouse button 1 is pressed over the Button. When **repeatdelay** or **repeatinterval** option is positive integer, this command is repeatedly called if mouse pointer is over the Button and until mouse button 1 is released.

-command

Specifies a Tcl command to associate with the Button. This command is typically invoked when mouse button 1 is released over the Button window.

-default

Specifies one of three states for the default ring: **normal**, **active**, or **disabled**. In active state, the button is drawn with the platform specific appearance for a default button. In normal state, the button is drawn with the platform specific appearance for a non-default button, leaving enough space to draw the default button appearance. The normal and active states will result in buttons of the same size. In disabled state, the button is drawn with the non-default button appearance without leaving space for the default appearance. The disabled state may result in a smaller button than the active state.

-disarmcommand
Specifies a Tcl command to associate with the Button when mouse button 1 is released. This command is called even if pointer is not over the Button, and always before the command specified by **command** option. It is typically used in conjuntion with **armcommand**, **repeatdelay** and **repeatinterval**.

-height
Specifies a desired height for the Button. If an image or bitmap is being displayed in the Button then the value is in screen units; for text it is in lines of text. If this option isn't specified, the Button's desired height is computed from the size of the image or bitmap or text being displayed in it.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-name
Specifies a standard name for the button. If the option ***nameName** is found in the resources database, then **text** and **underline** options are extracted from its value.

-relief
Specifies the 3-D effect desired for the widget. Acceptable values are standard values for button relief (**raised**, **sunken**, **flat**, **ridge**, **solid**, and **groove**) and **link**, which specifies that button relief is **flat** when pointer is outside the button and **raised** when pointer is inside.

-state
Specifies one of three states for the Button: **normal**, **active**, or **disabled**. In normal state the Button is displayed using the **foreground** and **background** options. The active state is typically used when the pointer is over the Button. In active state the Button is displayed using the **activeforeground** and **activebackground** options. Disabled state means that the Button should be insensitive: the default bindings will refuse to activate the widget and will ignore mouse button presses. In this state the **disabledforeground** and **background** options determine how the Button is displayed.

-underline
Specifies the integer index of a character to underline in the label of the button. 0 corresponds to the first character of the text displayed, 1 to the next character, and so on. The binding **<Alt-char>** is automatically set on the toplevel of the Button to call Button:: **setfocus**.

-width
If an image or bitmap is being displayed in the Button then the value is in screen units; for text it is in characters. If this option isn't specified, the Button's desired width is computed from the size of the image or bitmap or text being displayed in it.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* invoke**
If Button **state** is not disabled, this invoke the commands of the Button. Button is redisplayed with active color and sunken relief, and **armcommand** is called. Then Button is redisplayed with normal color and its defined relief, and **disarmcommand** then **command** are called.

invoke is called when Button has input focus and user press the space bar.

ButtonBox

Keywords: ButtonBox

NAME
ButtonBox - Set of buttons with horizontal or vertical layout

CREATION
ButtonBox *pathName* ?*option value...*?

WIDGET-SPECIFIC OPTIONS

-background or -bg	-default
-homogeneous	
-orient	
-padx	
-pady	
-spacing	
-state	

WIDGET COMMAND

pathName add ?*option value...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName delete *index*

pathName index *index*

pathName insert *index* ?*option value...*?

pathName invoke *index*

pathName itemcget *index option*

pathName itemconfigure *index* ?*option*? ?*value option value* ...?

pathName setfocus *index*

DESCRIPTION

ButtonBox layouts Button horizontally or vertically. Some commands take an *index* as argument indicating on which Button it work. This index may be specified in any of the following forms:

number
Specifies the Button numerically, where 0 corresponds to the first added Button, 1 to the next, and so on.

end or last
Indicates the last item added.

default
Indicates the default Button.

name
Indicates the button whose **-name** option is *name*.

text
Indicates the button whose **-text** option is *text*.

WIDGET-SPECIFIC OPTIONS

-background
Specifies a default background color for all added buttons and for the frame.

-default
Specifies the default button of the button box. The value is an integer referencing the n-th added button, starting from 0. If this value is -1 (the default), all button wil be drawn with their -default option set to disabled, and this value can not be changed. If this value is not -1, the associated button is drawn with -default option set to active and the others are drawn

with -default option set to normal. The value can be changed by configure.

-homogeneous (read-only)
Specifies wether or not buttons must have the same width for horizontal layout.

-orient (read-only)
Specifies the orientation of the button box. If this option is **horizontal** (the default), buttons are added from top to bottom. If this option is **vertical**, buttons are added from left to right.

-padx
Specifies a default value for the -padx option of all added buttons.

-pady
Specifies a default value for the -pady option of all added buttons.

-spacing
Specifies the default spacing between buttons. This value can be changed before each call to **add**.

-state
Specifies a state for all the buttons in the button box. Can be any state supported by buttons.

WIDGET COMMAND

***pathName* add ?*option value...*?**
Add a button to the button box.

See Button for description of options.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

pathName* delete *index
Delete the button at *index* from the button box.

pathName* index *index
Return the numerical index corresponding to the item.

***pathName* insert *index* ?*option value...*?**
Insert a new button into the button box before the given index.

pathName* invoke *index
Invoke the Button given by *index*.

pathName* itemcget *index option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *index* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* setfocus *index
Set the focus to the Button given by *index*.

ComboBox

Keywords: ComboBox

NAME
ComboBox - ComboBox widget

CREATION
ComboBox *pathName* ?*option value...*?

OPTIONS from ArrowButton

-background or -bg	-disabledforeground
-foreground or -fg	-state

OPTIONS from Entry

-command	-disabledforeground
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-droptypes	-editable
-entrybg (see -background)	-entryfg (see -foreground)
-exportselection	-font
-helptext	-helptype
-helpvar	-highlightbackground

-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

WIDGET-SPECIFIC OPTIONS

-autocomplete	-autopost
-bwlistbox	-expand
-height	-hottrack
-images	-listboxwidth
-modifycmd	-postcommand
-values	

WIDGET COMMAND

pathName bind ?*arg...*?

pathName cget *option*

pathName clearvalue

pathName configure ?*option*? ?*value option value* ...?

pathName get

pathName getlistbox

pathName getvalue

pathName icursor *index*

pathName post

pathName setvalue *index*

pathName unpost

DESCRIPTION

ComboBox widget enables the user to select a value among a list given by the **values** option. The list of possible values can be popped by pressing the ArrowButton or by clicking in the entry when **editable** value of the ComboBox is **false**.
If **editable** value of the ComboBox is **true** and the entry has the focus, the user can press the top and bottom arrow keys to modify its value. If the current value exactly match a value in the list, then the previous (for top arrow key) or then next (for bottom arrow key) value in the list is displayed. If the current value match the beginning of a value in the list, then this value is displayed. If the current value doesnt match anything, then the first value is displayed.

WIDGET-SPECIFIC OPTIONS

-autocomplete
Specifies whether or not the combobox should attempt to auto-complete the value in the entry field as the user types. If true, the combobox will fill in a value that it finds in its value list as the user types into the entry.

-autopost
Specifies whether or not the combobox should post the drop down as the user types. If true, the combobox will post and scroll to the entry that most closely matches the user entry.

-bwlistbox
Specifies that the combobox should use a BWidget listbox in its drop down instead of the standard Tk option. This option is enabled by default if the **-images** option is not empty.

-expand
Specifies the value expansion behavior for the window. It must be **none** (default) or **tab**. If **tab** is specified, then a Tab binding is added to attempt to expand the current value based on the other values in **-values**.

-height
Specifies the desired height for the window, in lines. If zero or less, then the desired height for the window is made just large enough to hold all the elements in the listbox.

-modifycmd
Specifies a Tcl command called when the user modify the value of the ComboBox by selecting it in the listbox or pressing arrow key.

-postcommand
Specifies a Tcl command called before the listbox of the ComboBox is mapped.

-values
Specifies the values to display in the listbox of the ComboBox.

WIDGET COMMAND

***pathName* bind ?*arg...*?**
Set bindings on the entry widget.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* clearvalue**
Clears the current text of the ComboBox.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* get**
Returns the current contents of the entry.

***pathName* getlistbox**
Returns the path to the listbox in the drop down.

***pathName* getvalue**
Returns the index of the current text of the ComboBox in the list of values, or -1 if it doesn't match any value.

pathName* icursor *index
Arrange for the insertion cursor to be displayed just before the character given by *index*. Returns an empty string.

***pathName* post**
Post the drop down.

pathName* setvalue *index
Set the text of the ComboBox to the value indicated by *index* in the list of values. *index* may be specified in any of the following forms:

last
Specifies the last element of the list of values.

first
Specifies the first element of the list of values.

next
Specifies the element following the current (ie returned by **getvalue**) in the list of values.

previous
Specifies the element preceding the current (ie returned by **getvalue**) in the list of values.

@*number*
Specifies the integer index in the list of values.

***pathName* unpost**

Unpost the drop down.

BINDINGS

When Entry of the ComboBox has the input focus, it has the following bindings, in addition to the default Entry bindings:

- Page up set the value of the ComboBox to the first value.
- Page down set the value of the ComboBox to the last value.
- Arrow up set the value of the ComboBox to the previous value.
- Arrow down set the value of the ComboBox to the next value.
- If **-autopost** is enabled, Escape unposts the listbox.

If the listbox is not mapped and ComboBox is not editable or disabled, mouse button 1 on the Entry cause the listbox to popup, as if the user press the ArrowButton.

Dialog

Keywords: Dialog

NAME

Dialog - Dialog abstraction with custom buttons

CREATION

Dialog *pathName* ?*option value...*?

OPTIONS from ButtonBox

-background or -bg	-homogeneous
-padx	-pady
-spacing	

WIDGET-SPECIFIC OPTIONS

-anchor
-bitmap
-cancel
-default
-geometry
-image
-modal
-parent
-place

-separator
-side
-title
-transient

WIDGET COMMAND

pathName add ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName draw ?*focus*?

pathName enddialog *result*

pathName getframe

pathName invoke *index*

pathName itemcget *index option*

pathName itemconfigure *index* ?*option*? ?*value option value* ...?

pathName setfocus *index*

pathName withdraw

DESCRIPTION

Dialog widget enables the user to create a dialog box. Some commands take an *index* as argument indicating on which Button it work. This index is the same specified for equivalent ButtonBox command:

number
Specifies the Button numerically, where 0 corresponds to the first added Button, 1 to the next, and so on.

end or last
Indicates the last item added.

default
Indicates the default Button.

WIDGET-SPECIFIC OPTIONS

-anchor (read-only)
Specifies the anchor point of the ButtonBox. Must be one of **w**, **e**, **n**, **s** or **c**. If **side** option is set to *top* or *bottom*, **anchor** values *n*, *s* and *c* have the same effect. If **side** option is set to *left* or *right*, **anchor** values *w*, *e* and *c* have the same effect.

-bitmap (read-only)
Specifies a bitmap to display at the left of the user frame. **image** option override **bitmap**.

-cancel
Specifies the number of the cancel button of the Dialog. When user presses Esc or the windows close button, this button is invoked. If set to **-1**, Esc does not invoke anything. The window close button *destroys* the dialog in this case and returns **-1**.

-default
Specifies the number of the default button of the Dialog. When user press Return in the Dialog, this button is invoked.

-geometry
Set the **geometry** used when method **draw** is called. See **wm geometry** for a parameter description.

-image (read-only)
Specifies an image to display at the left of the user frame. **image** option override **bitmap**.

-modal
This option must be **none**, **local** or **global**. The value of this option specifies the grab mode of the dialog and how works Dialog::**draw**.

-parent
Parent of the Dialog. Dialog is placed relative to its parent. If empty, it is placed relative to the root window. Also see **place** option.

-place
Specifies where to draw the Dialog toplevel relative to the dialog's parent. Must be one of **none**, **center**, **left**, **right**, **above**, **below**. Default value of **place** is *center*.

-separator (read-only)
Specifies wether or not to draw a separator between the user frame and the ButtonBox.

-side (read-only)
Specifies where to draw the ButtonBox relative to the user frame. Must be one of **top**, **left**, **bottom** or **right**.

-title
Title of the Dialog toplevel.

-transient (read-only)
Specifies if the Dialog Toplevel should be a transient window or not. Default value of **transient** is *true*.

WIDGET COMMAND

***pathName* add ?*arg...*?**
Add a button to the button box of the dialog box. Default -command option is

Dialog::enddialog $path index where *index* is number of button added.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* draw ?*focus*?**
This command draw the Dialog, and set grab to it following **modal** option. If **modal** option is set to *none*, the command returns immediatly an empty string. In all other case, the command returns when Dialog::**enddialog** is called or when Dialog is destroyed. The return value is the result argument of Dialog::**enddialog** or -1 if it is destroyed.

By default, the focus is set to the default button referenced by **default** option, or to the toplevel of Dialog if no default button has been set. If *focus* is present, it must be a pathname, or an index to a button. Initial focus is set on this pathname or corresponding button.

pathName* enddialog *result
This command is typically called within a command of a button to make Dialog::**draw** return.

***pathName* getframe**
Returns the pathname of the user window.

pathName* invoke *index
Invoke the Button given by *index*.

pathName* itemcget *index option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *index* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* setfocus *index
Set the focus to the Button given by *index*.

***pathName* withdraw**
Call this command to hide the dialog box.

DragSite

Keywords: DragSite

NAME

DragSite - Commands set for Drag facilities

COMMAND

DragSite::include *class type event*

DragSite::register *path* ?*option value...*?

DragSite::setdrag *path subpath initcmd endcmd* ?*force*?

DESCRIPTION

Commands of this namespace enable user to define a BWidget or a Tk widget as a drag site.

COMMAND

DragSite::include *class type event*

This command provides a simple way to include options relatives to a drag site into BWidget resources definition. It includes the options needed for **register**: *-dragevent*, initialized to *event*, *-draginitcmd* and *-dragendcmd*, initialized to empty string, and two new options:

-dragenabled	Specifies wether or not drag is active (initialized to 0)

DragSite::register *path ?option value...?*

This command is used to declare *path* as a drag site. Options are:

-dragendcmd

Command called when drag terminates (ie when user release drag icon). This command is called with the following arguments:
- the pathname of the drag source (the widget itself),
- the pathname of the drop target,
- the operation,
- the type of the dragged data,
- the dragged data,
- result of the drop (result of the call to **-dropcmd** of the target),

If the drop does not occurs, the target and the operation are empty string and the result is 0.

-dragevent

Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd

Command called when drag initiates. When the event of option **dragevent** occurs on *path*, this command is called with the following arguments:
- pathname of the drag source (*path*),

- root x-coordinate of pointer,
- root y-coordinate of pointer,
- a toplevel created to represent dragged data. When returning, if it has no children, a bitmap is automatically displayed.

If the command returns an empty string, then the drag will be suppressed. Otherwise the command must return a list containing three elements:
- the type of the data,
- the list of acceptable basic operations (**copy**, **move** and **link**)
- and the data.

Note that even if **copy** does not appear in the list of basic operation, it is considered as an acceptable operation, since **copy** semantic does not modify the drag source.

DragSite::setdrag *path subpath initcmd endcmd ?force?*

This command provides a simple way to call **register** during a BWidget creation or configuration.
- *path* is the pathname of the BWidget,
- *subpath* is the pathname of the tk widget where drag event occurs,
- *initcmd* BWidget command for *drag-init* event,
- *endcmd* BWidget command for *drag-end* event,
- *force* specifies wether or not to call **register** whenever no option value has changed (0 by default - for BWidget configuration, use 1 for BWidget creation).

setdrag verifies the modification flag of options **dragenabled** and **dragevent** and calls **register** if needed according to the options values and *initcmd* and *endcmd* arguments. **draginitcmd** and **dragendcmd** are not taken from options of widget because they are considered as user command, called by BWidget implementation of *drag-init* and *drag-end* events.

DropSite

Keywords: DropSite

NAME

DropSite - Commands set for Drop facilities

COMMAND

DropSite::include *class types*

DropSite::register *path* ?*option value...*?

DropSite::setcursor *cursor*

DropSite::setdrop *path subpath dropover drop* ?*force*?

DropSite::setoperation *op*

DESCRIPTION

Commands of this namespace enable user to define a BWidget or a Tk widget as a drop site. A drop site is composed of the type of object that can be dropped and associated operation, a command called when drop occurs, and a command when an object is dragged over the widget. A drop site must have at least one type of acceptable object and a drop command.

COMMAND

DropSite::include *class types*

This command provides a simple way to include options relatives to a drop site into BWidget resources definition. It includes the options needed for **register**, *-dropovercmd* and *-dropcmd* , initialized to empty string, and *-droptypes*, initialized to *types*, and one new option:

-dropenabled	Specifies wether or not drop is active (initialized to 0)

DropSite::register *path* ?*option value...*?

This command is used to declare *path* as a drop site. Options are:

-dropcmd

This command is called when user release the drag icon over a valid drop target widget. Arguments passed to the command are:
- pathname of the drop target (the widget itself),
- pathname of the drag source,
- root x-coordinate of the pointer,
- root y-coordinate of the pointer,
- operation,
- type of the dragged data,
- dragged data.

Its return values is passed as a result to the **-dragendcmd** command of the drag source widget.

-dropovercmd

This command can be used to provide a dynamic drag while *drag-over* events. While a drag occurs, the events <Enter>, <Motion> and <Leave> are caught and handled internally. Arguments passed to the command are:
- pathname of the drop target (the widget itself),
- pathname of the drag source,
- event over the drop target: *enter*, *motion* or *leave*,
- root x-coordinate of the pointer,
- root y-coordinate of the pointer,
- operation,
- type of the dragged data,
- dragged data.

Command must the new status of the drag:
- 0 if widget refuse this drag. Command will not be recalled on motion/leave event.
- 1 if widget accept this drag. Command will not be recalled on motion/leave event.
- 2 if widget refuse this drag. Command will be recalled on each motion event to

reevaluate.
- 3 if widget accept this drag. Command will be recalled on each motion event to reevaluate.

Here is a list of events and associated actions on a DropSite widget. This example assumes that dragged data type is valid for the drop target. **status** is the status of the drag on a DropSite. Its value is:

Event	Old status	Action	New status
<Enter>	-	if DropSite has dropovercmd, call it with enter	result of dropovercmd
else	1		
<Motion>	0 or 1		unchanged
2 or 3	call dropovercmd with motion	result of dropovercmd	
<Leave>	0 or 1		-
2 or 3	call dropovercmd with leave	-	
<Drop>	0	call dragendcmd of drag source	-
1	call dropcmd and call dragendcmd of drag source		
2	call dropovercmd with leave and call dragendcmd of drag source		
3	call dropcmd and call dragendcmd of drag source		

-droptypes

Specifies a list {*type oplist* ?*type oplist*? ...} of acceptable types and associated operations for the drop target. For each type, *oplist* is a list {*descops mod* ?*descops mod*? ...} describing operations and modifier keys for these operations. *descops* describe an operation. It can be a predefined operations (**copy**, **move** or **link**) or a new user defined operation, of the form {*subop* **baseop** ?*bitmap*?}. *subop* is the name given to the sub operation, *baseop* is the name of the base operation (**copy**, **move** or **link**) and *bitmap* is a bitmap to display for the operation. If *bitmap* is empty, the default bitmap of the base operation is used for the sub operation. *subop* can be a base operation, in order to change

the bitmap of a base operation. In this case, *baseop* must be empty or equal to *subop*. *mod* is the modifer key for the operation. It can be:
- **none** to specify that no modifier key is pressed. This modifier can only be used with a sub operation named **default** (and vice versa), which has the behaviour of not display any bitmap operation. For all type, if the modifier **none** is not given, it is automatically associated to the **default** sub operation of a **copy** base operation.
- **program** to specifies a sub operation accessible only by **DropSite::setoperation**.
- A list combining **shift**, **control** and **alt**, which means their corresponding key.

DropSite::setcursor *cursor*

This command can be used within the script **dragovercmd**. It is usefull to provide visual effect about the state of the drag.

DropSite::setdrop *path subpath dropover drop* ?*force*?

This command provides a simple way to call **register** during a BWidget creation or configuration.
- *path* is the pathname of the BWidget,
- *subpath* is the pathname of the tk widget where drag event occurs,
- *dropover* is a command for *drag-over* event,
- *drop* is a command for *drop* event,
- *force* specifies wether or not to call **register** whenever no option value has changed (0 by default - for BWidget configuration, use 1 for BWidget creation).

setdrop verifies the modification flag of options **dropenabled** and **droptypes** and calls **register** if needed according to the options values and *dropover* and *drop* arguments. **dropovercmd** and **dropcmd** are not taken from options of widget because they are considered as user command, called by BWidget implementation of *drag-over* and *drop* events.

DropSite::setoperation *op*

Description text

DynamicHelp

Keywords: DynamicHelp

NAME

DynamicHelp - Provide help to Tk widget or BWidget

COMMAND

DynamicHelp::add *widget* ?*option value ...*?

DynamicHelp::configure ?*option*? ?*value option value* ...?

DynamicHelp::delete *widget*

DynamicHelp::include *class type*

DynamicHelp::register *path type* ?*arg...*?

DynamicHelp::sethelp *path subpath* ?*force*?

DESCRIPTION

Description text

COMMAND

DynamicHelp::add ?*option value ...*?

This command adds dynamic help to the given *widget*.

-command
If specified, refers to a command to execute to get the help text to display. The command must return a string to display. If the command returns an empty string, no help is displayed.

-index
If specified, refers to a menu index to bind the help to instead of a widget. If **-type** is not menu, this option is ignored.

-item
If specified, refers to an item or tag on a canvas widget or to a tag in a text widget to bind the help to instead of to a widget.

-type
Specifies the type of help. Can be: **balloon**, **variable** or **menu**. Default is **balloon**.

-text
The text to be displayed as help.

-variable
Specifies a variable name to which the help string will be written. Some other widget (e.g., a status bar) is responsible for displaying this variable.

Creating dynamic help for a menu is a two-step process. The menu itself must first be added and then each menu entry must be added separately. Here is a brief example.

```
# create menu
menu .m -type menubar
# associate menubar to toplevel BEFORE DynamicHelp::register
# to make it works with menu clone name
. configure -menu .m
.m add cascade -label "File" -menu .m.file
menu .m.file
.m.file add command -label "Open..."
.m.file add command -label "Quit"
# create label for help, using variable varinfo
label .l -textvariable varinfo
# associate all entries of menu .m.file to variable varinfo
DynamicHelp::add .m.file -type menu -variable varinfo
# then declare entries of .m.file
DynamicHelp::add .m.file -type menu -index 0 -text "Detach menu"
DynamicHelp::add .m.file -type menu -index 1 -text "Open a file"
DynamicHelp::add .m.file -type menu -index 2 -text "Exit demo"
```

DynamicHelp::configure ?*option*? ?*value option value ...*?

This command configure the ballon help.

-borderwidth

Width of the black border around the balloon.

-delay

Define the delay in millisecond of mouse inactivity before displaying the balloon.

-state

Specifies one of two states for help balloons: **normal** and **disabled**.

If **state** is **disabled**, help balloons will not be displayed for any registered widget.

-topbackground

The background color of the toplevel window created for a balloon.

Other standard options are:

-background or -bg	-font
-foreground or -fg	-justify
-padx	-pady

DynamicHelp::delete *widget*

Delete all dynamic help for the given *widget*.

DynamicHelp::include *class type*

Description text

DynamicHelp::register *path type* ?*arg...*?

Its use is deprecated. Use DynamicHelp::add instead.

Register a help text to the widget *path*. *type* determines the type of the help or the type of the widget. Depending on *type*, other options must be provided.

type	options
balloon	?tagOrItem? text
variable	?tagOrItem? varName text
menu	varName
menuentry	index text

If one of the option is missing or is empty, help is removed for this widget.

If *tagOrItem* is specified, then *path* is a canvas or a text. In case of a canvas, *tagOrItem* is the name of a tag or item on the canvas to which the help will be bound. In case of a text, *tagOrItem* is the name of a tag on the text to which the help will be bound.

For type other than *balloon*, *varName* is typically a variable linked to a label. For menu,

balloon type help is not available. To declare a help for menu, you first declare the menu, and then entries of this menu. For example:

create menu menu .m -type menubar # associate menubar to toplevel BEFORE DynamicHelp::register # to make it works with menu clone name. configure -menu .m .m add cascade -label "File" -menu .m.file menu .m.file .m.file add command -label "Open..." .m.file add command -label "Quit" # create label for help, using variable varinfo label .l -textvariable varinfo # associate all entries of menu .m.file to variable varinfo DynamicHelp::register .m.file menu varinfo # then declare entries of .m.file DynamicHelp::register .m.file menuentry 0 "Detach menu" DynamicHelp::register .m.file menuentry 1 "Open a file" DynamicHelp::register .m.file menuentry 2 "Exit demo"

Notice that if popup menu is owned by a menubar, you must associate first the menubar to its toplevel. In this case, when you create a menu popup, its clone window is also created, and DynamicHelp::register detects the exitence of the clone window and maps events to it.

DynamicHelp::sethelp *path subpath* ?*force*?

Description text

Entry

Keywords: Entry

NAME

Entry - Entry widget with **state** option, dynamic help and drag and drop facilities

CREATION

Entry *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-disabledbackground	-disabledforeground
-exportselection	-font
-foreground or -fg	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-relief	-selectbackground
-selectborderwidth	-selectforeground
-takefocus	-text
-textvariable	-xscrollcommand

WIDGET-SPECIFIC OPTIONS

-command	-dragenabled
-dragendcmd	-dragevent
-draginitcmd	-dragtype
-dropcmd	-dropenabled
-dropovercmd	-droptypes
-editable	-helptext
-helptype	-helpvar
-show	-state
-width	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName invoke

DESCRIPTION

The **Entry** widget extends the default Tk entry. Options have been added to provide visual effect depending on the state of the Entry, DynamicHelp options, and Drag and Drop. Entry behaves much like a Label, with **text** option to set its contents.
Tk entry command can also be used on Entry widget.

WIDGET-SPECIFIC OPTIONS

-command
Specifies a command when user press <Return> in the Entry.

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**. If **dragendcmd** is empty, the internal *dragend* command updates the entry following the operation (**move** or **copy**) and the dragged data (whole or selected part of the entry).

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd

Specifies a command to be called when **dragevent** occurs on widget. **draginitcmd** must be a command conforming to the description of the option **draginitcmd** of **DragSite::register**. if **draginitcmd** is empty, the command refuse the drag if entry is empty or if portion of text is selected and event doesn't occur above the selection. In all other cases, the command returns:
- as the data type, the value of option **dragtype** or *TEXT* if empty,
- as the operations, *{copy move}* if **state** is normal and **editable** is true, or *{copy}* only in other cases,
- as the data, the whole content or the selected portion of the entry.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
Entry has a command wrapper for *drop* events. This command stops auto scrolling and extract current position. If **dropcmd** is not empty, it is called with the following arguments:
- the pathname of the Entry,
- the pathname of the drag source,
- the numeric index in the entry designated by the cursor,
- the current operation,
- the data type,
- the data.

and must return a value conforming to dropcmd option described in DropSite::register. If dropcmd is empty, the wrapper updates the entry following the type of data:

COLOR or FGCOLOR	reconfigure the foreground of the Entry
BGCOLOR	reconfigure the background of the Entry
TEXT, or any other tag	reconfigure the Entry to display the associated string.

and returns 1.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
Entry has a command wrapper for *drag-over* events. This command enables auto scrolling and position extraction during the *drag-over*. If **dropovercmd** is empty, the wrapper accepts the drop if **editable** option is true and **state** option is normal. If **dropovercmd** is not empty, it is called with the following arguments:
- the pathname of the Entry,
- the pathname of the drag source,
- the event,
- the numeric index in the entry designated by the cursor,
- the current operation,

- the data type,
- the data.

and must return a value conforming to **dropovercmd** option described in **DropSite::register**.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default accepts *FGCOLOR*, *COLOR*, *BGCOLOR* and *TEXT*, all with **copy** and **move** operations.

-editable
Specifies whether the Entry is editable by the user. Equivalent to the **state** option of the Tk entry widget.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-show
If this option is specified, then the true contents of the entry are not displayed in the window. Instead, each character in the entry's value will be displayed as the first character in the value of this option, such as ``*''. This is useful, for example, if the entry is to be used to enter a password. If characters in the entry are selected and copied elsewhere, the information copied will be what is displayed, not the true contents of the entry.

-state
Specifies one of two states for the Entry: **normal** or **disabled**. In normal state the text of the Entry is displayed using the **foreground** option. In disabled state the text of the Entry is displayed using the **disabledforeground** option. If the entry is disabled then the value may not be changed by user input and no insertion cursor will be displayed, even if the input focus is in the widget. Disabled state is the same as not editable with visual effect.

-width
Specifies an integer value indicating the desired width of the entry window, in average-size characters of the widget's font. If the value is less than or equal to zero, the widget picks a size just large enough to hold its current text.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then

the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* invoke**

Calls the command specified by the option **-command**.

Label

Keywords: Label

NAME

Label - Label widget with **state** option, dynamic help and drag and drop facilities

CREATION

Label *pathName* ?*option value...*?

STANDARD OPTIONS

-anchor
-background or -bg
-bitmap
-borderwidth or -bd
-cursor
-disabledforeground
-font
-foreground or -fg
-highlightbackground
-highlightcolor
-highlightthickness
-image
-justify
-padx
-pady
-relief
-takefocus
-text

-textvariable
-wraplength

WIDGET-SPECIFIC OPTIONS

-dragenabled
-dragendcmd
-dragevent
-draginitcmd
-dragtype
-dropcmd
-dropenabled
-dropovercmd
-droptypes
-focus
-height
-helptext
-helptype
-helpvar
-name
-state
-underline
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName setfocus

DESCRIPTION

The **Label** widget extends the default Tk label. Options have been added to provide visual effect

depending on the state of the Label, DynamicHelp options, and Drag and Drop.

WIDGET-SPECIFIC OPTIONS

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**.

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd
Specifies a command to be called when **dragevent** occurs on widget. **draginitcmd** must be a command conforming to the description of the option **draginitcmd** of **DragSite::register**. If **draginitcmd** is empty, the internal **draginitcmd** command is used instead and returns:

```
|   |   |
| --- | --- |
| *IMAGE {copy} \<image name>*  | if an image is displayed.  |
```

Note that if **dragtype** option is not empty, its value is used instead of those above.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
Specifies a command to be called when drop occurs on the widget. **dropcmd** must be a command conforming to the description of the option **dropcmd** of **DropSite::register**. If **dropcmd** is empty, the command updates the label following the type of the data:

```
|   |   |
| --- | --- |
| *COLOR* or *FGCOLOR* | reconfigure the **foreground** of the Label. |
| *BGCOLOR* | reconfigure the **background** of the Label. |
| *IMAGE* | reconfigure the Label to display the associated image. |
| *BITMAP* | reconfigure the Label to display the associated bitmap. **image**
    option is set to empty. |
| *TEXT*,   or any other tag | reconfigure the Label to display the associated
    string. **image** and **bitmap** options are set to empty. |
```

and returns 1.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
Specifies a command to be called when drag icon is over the widget. **dropovercmd** must be a command conforming to the description of the option **dropovercmd** of **DropSite::register**. If **dropovercmd** is empty, Label always accepts the drop if data type is *FGCOLOR*, *COLOR*, *BGCOLOR*, and accepts all other data type only if **state** is normal.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default accepts *FGCOLOR*, *COLOR*, *BGCOLOR*, *TEXT*, *BITMAP* and *IMAGE*, all with **copy** and **move** operations.

-focus
Specifies a pathname to set the focus on for Label::**setfocus** command.

-height
Specifies a desired height for the label. If an image or bitmap is being displayed in the label then the value is in screen units, for text it is in lines of text. If this option isn't specified, the label's desired height is computed from the size of the image or bitmap or text being displayed in it.

-helptext
Text for dynamic help. If empty, no help is available for this widget. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-name
Specifies a standard name for the label. If the option ***nameName** is found in the resource database, then **text** and **underline** options are extracted from its value.

-state
Specifies one of two states for the Label: **normal** or **disabled**. In normal state the text of the Label is displayed using the **foreground** option. In disabled state the text of the Label is displayed using the **disabledforeground** option.

-underline
Specifies the integer index of a character to underline in the label. 0 corresponds to the first character of the text displayed, 1 to the next character, and so on. The binding **<Alt-char>** is automatically set on the toplevel of the Label to call Label::**setfocus**.

-width
Specifies a desired width for the label. If an image or bitmap is being displayed in the label then the value is in screen units, for text it is in characters. If this option isn't specified, the label's desired width is computed from the size of the image or bitmap or text being displayed in it.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the

corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* setfocus**
Set the focus on the pathname given by **-focus** option if **-state** is *normal*.

LabelEntry

Keywords: LabelEntry

NAME
LabelEntry - LabelFrame containing an Entry widget.

CREATION
LabelEntry *pathName* ?*option value...*?

OPTIONS from Entry

-borderwidth or -bd	-command
-disabledforeground	-dragenabled
-dragendcmd	-dragevent
-draginitcmd	-dragtype
-dropcmd	-dropenabled
-dropovercmd	-droptypes
-editable	-entrybg (see -background)
-entryfg (see -foreground)	-exportselection
-font	-helptext
-helptype	-helpvar
-highlightbackground	-highlightcolor
-highlightthickness	-insertbackground
-insertborderwidth	-insertofftime
-insertontime	-insertwidth
-justify	-relief
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus

-text	-textvariable
-width	-xscrollcommand

OPTIONS from LabelFrame

-background or -bg	-disabledforeground
-foreground or -fg	-helptext
-helptype	-helpvar
-label (see -text)	-labelanchor (see -anchor)
-labelfont (see -font)	-labelheight (see -height)
-labeljustify (see -justify)	-labelwidth (see -width)
-name	-padx
-pady	-side
-state	-underline
-wraplength	

WIDGET COMMAND

pathName bind ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

LabelEntry is a widget composed of LabelFrame widget containing an Entry widget. Tk entry command can also be used on LabelEntry widget.

WIDGET COMMAND

***pathName* bind ?*arg...*?**

Set bindings on the entry widget.

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then

the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

LabelFrame

Keywords: LabelFrame

NAME

LabelFrame - Frame with a Label

CREATION

LabelFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-borderwidth or -bd	-relief

OPTIONS from Label

-anchor	-background or -bg
-bitmap	-disabledforeground
-focus	-font
-foreground or -fg	-height
-helptext	-helptype
-helpvar	-image
-justify	-name
-padx	-pady
-state	-text
-textvariable	-underline
-width	-wraplength

WIDGET-SPECIFIC OPTIONS

-side

WIDGET COMMAND

LabelFrame::align ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

DESCRIPTION

LabelFrame enables user to create a frame with a Label positionned at any side. LabelFrame is used by ComboBox and SpinBox.

WIDGET-SPECIFIC OPTIONS

-side (read-only)
Specifies where to position the Label relative to the user frame: **top**, **bottom**, **left** or **right**.

WIDGET COMMAND

LabelFrame::align ?*arg...*?
This command align label of all widget given by *args* of class LabelFrame (or "derived") by setting their width to the max one +1

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Return the frame where the user can create any other widget.

ListBox

Keywords: ListBox

NAME
ListBox - ListBox widget

CREATION
ListBox *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-cursor	-highlightbackground

-highlightcolor	-highlightthickness
-relief	-selectbackground
-selectforeground	-takefocus
-xscrollcommand	-yscrollcommand

WIDGET-SPECIFIC OPTIONS

-autofocus	-deltax	-deltay
-dragenabled	-dragendcmd	
-dragevent	-draginitcmd	
-dragtype	-dropcmd	
-dropenabled	-dropovercmd	
-dropovermode	-droptypes	
-height	-multicolumn	
-padx	-redraw	
-selectfill	-selectmode	-width

WIDGET COMMAND

pathName bindImage *event script*

pathName bindText *event script*

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName delete ?*arg...*?

pathName edit *item text* ?*verifycmd*? ?*clickres*? ?*select*?

pathName exists *item*

pathName index *item*

pathName insert *index item* ?*option value...*?

pathName item *first* ?*last*?

pathName itemcget *item option*

pathName itemconfigure *item* ?*option*? ?*value option value* ...?

pathName items ?*first*? ?*last*?

pathName move *item index*

pathName reorder *neworder*

pathName see *item*

pathName selection *cmd* ?*arg...*?

pathName xview ?*arg...*?

pathName yview ?*arg...*?

DESCRIPTION

ListBox widget uses canvas to display a list of items. Each item is composed of a label with its own font and foreground attributes, and an optional image or window. Each item is drawn in a single line, whose height is defined by the **deltay** option, so they must have at most this height. A item is uniquely identified by a string given at creation (by the **insert** command). The ListBox can have one or more columns, depending on **multicolumn** option. The user do not handle columns; the number of columns is determined following the height of the ListBox in order to see each item vertically.

WIDGET-SPECIFIC OPTIONS

-autofocus
If this option is true, the listbox will take focus any time the user clicks in it. Without focus, the listbox's mouse wheel bindings will not work properly. The default is true.

-deltax
Specifies horizontal pad between each columns.

-deltay
Specifies vertical size of the items.

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**.

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd

ListBox has a command wrapper for *drag-init* events. This command refused the drag if no item is designated. In other cases: If **draginitcmd** is empty, it returns:
- the value of option **dragtype** or *LISTBOX_ITEM* if empty as the data type,
- *{move copy link}* as the operations,
- the item identifier as the data.

If **draginitcmd** is not empty, it is called with the following arguments:
- the pathname of the listbox,
- the identifier of the dragged item,
- the toplevel created to represent dragged data.

and must return a value conforming to **draginitcmd** option described in **DragSite::register**.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
ListBox has a command wrapper for *drop* events. This command stops auto scrolling and extract item and position. If **dropcmd** is not empty, it is called with the following arguments:
- the pathname of the listbox,
- the pathname of the drag source,
- a list describing where the drop occurs. It can be:
- the current operation,
- the data type,
- the data.
  - *{***widget***}*,
  - *{***item** *item}* or
  - *{***position** *index}*.

The default drop command allows for drag-and-drop within the listbox but not to or from other widgets.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
LsitBox has a command wrapper for *drag-over* events. This command enables auto scrolling and position extraction during the *drag-over*. If **dropovercmd** is not empty, the command is called with the following aguments:
- the pathname of the listbox,
- the pathname of the drag source,
- a list describing where the drop can occur, whose elements are:
- the current operation,
- the data type,
- the data.
  - the string *widget* if **dropovertype** option contains *w*, else empty string.
  - the targeted item if drag icon points an item and **dropovertype** option contains *i*, else empty string.
  - an index within two items where drag icon points to if **dropovertype** option contains *p*,

else empty string.
- optionally, the preferred method if drop can occur both inside an item and between two items. The value is *position* or *item*.

The command must return a list with two elements:
- the drop status, conforming to those described in **dropovercmd** option of **DropSite::register**,
- the choosen method: *widget*, *item* or *position*.

-dropovermode
Specifies the type of *drop-over* interaction. Must be a combination of **w**, which specifies that drop can occurs everywhere on widget, **p**, which specifies that drop can occurs between two items, and **i**, which specifies that drop occurs inside items.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default is *LISTBOX_ITEM* with operations **copy** and **move**.

-height
Specifies the desired height for the listbox in units of **deltay** pixels.

-multicolumn
Specifies wether or not ListBox layouts items in order to see each one vertically.

-padx
Specifies distance between image or window and text of the items.

-redraw
Specifies wether or not the listbox should be redrawn when entering idle. Set it to false if you call **update** while modifying the listbox.

-selectfill
If true, the listbox will draw a selection rectangle that fills the listbox from left-to-right instead of just drawing a box around the selected item. This more closely mimics the standard Tk listbox.

-selectmode
Specifies the desired selection-mode for the listbox. Must be one of **none**, **single** or **multiple** . **selectmode** *single* allows to select 1 item by its text or image. **selectmode** *multiple* allows to select multiple items by their text or image. For more info on selectmodes *single* or *multiple*, see the Tk **listbox** command. Default value for **selectmode** is *none*.

-width
Specifies the desired width for the listbox in units of 8 pixels.

WIDGET COMMAND

pathName* bindImage *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the image of a item. The item idenfier on which the event occurs is appended to the command.

Any occurrence of **%W** in *script* is substituted with the path of the listbox.

pathName* bindText *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the label of a item. The item idenfier on which the event occurs is appended to the command.

Any occurrence of **%W** in *script* is substituted with the path of the listbox.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete ?*arg...*?**
Deletes all items in *arg*. *arg* can be a list of items or a list of list of items. To delete all items, do *$pathName delete [$pathName items]*.

***pathName* edit *item text* ?*verifycmd*? ?*clickres*? ?*select*?**
Provides a way for the user to edit in place the label of an item. The command takes the initial text as argument and does not modify the label of the edited node, but returns an empty string if edition is canceled, or the typed text if edition is accepted. When editing, the user can cancel by pressing Escape, or accept by pressing Return. *clickres* specifies what to do if the user click outside the editable area. If *clickres* is 0 (the default), the edition is canceled. If *clickres* is 1, the edition is accepted. In all other case, the edition continues. If edition is accepted and *modifycmd* is not empty, then it is called with the new text as argument and must return 1 to accept the new text, 0 to refuse it and continue edition. *select* specifies wether or not the initial text should be selected. Default is 1.

pathName* exists *item
Returns 1 if *item* exists in the listbox, else 0.

pathName* index *item
Returns the position of *item* in the list.

***pathName* insert *index item* ?*option value...*?**
Inserts a new item identified by *item* in the list at position *index*.

Any instance of *#auto* within the item name will be replaced by the number of the item in the order of insertion.

-data
User data associated to the item.

-fill
Specifies the foreground color of the label of the item.

-font
Specifies a font for the label of the item.

-image
Specifies an image to display at the left of the label of the item. **window** option override **image**.

-indent
Specifies the amount of extra space in pixels at the left of the item.

-text
Specifies the label of the item.

-window
Specifies a pathname to display at the left of the label of the item. **window** option override **image**.

***pathName* item *first* ?*last*?**
Its use is deprecated. Use items instead. If *last* is omitted, returns the item at index *first* in the list, or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the items between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

pathName* itemcget *item option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *item* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

***pathName* items ?*first*? ?*last*?**
If *first* and *last* are omitted, returns the list of all items. If *first* is specified and *last* omitted, returns the item at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the items between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

pathName* move *item index
Moves *item* at position *index* in the list.

pathName* reorder *neworder
Modifies the order of items in the listbox given by *neworder*. Items that do not appear in *neworder* are no moved.

pathName* see *item
Arrange the scrolling area to make *item* visible.

***pathName* selection *cmd* ?*arg...*?**

Modifies the list of selected items following *cmd*:

clear
remove all items of the selection.

set
set the selection to all items in *arg*

add
add all items of *arg* in the selection

remove
remove all items of *arg* of the selection

get
return the current selected items

***pathName* xview ?*arg...*?**

Standard command to enable horizontal scrolling of *pathName*.

***pathName* yview ?*arg...*?**

Standard command to enable vertical scrolling of *pathName*.

BINDINGS

A **<<ListboxSelect>>** virtual event is generated any time the selection in the listbox changes.

The listbox has all the standard mouse wheel bindings when it has focus.

MainFrame

Keywords: MainFrame

NAME

MainFrame - Manage toplevel with menu, toolbar and statusbar

CREATION

MainFrame *pathName* ?*option value...*?

OPTIONS from ProgressBar

-background or -bg	-progressfg (see -foreground)
-progressmax (see -maximum)	-progresstype (see -type)
-progressvar (see -variable)	

WIDGET-SPECIFIC OPTIONS

-height
-menu
-separator
-textvariable

-width

WIDGET COMMAND

pathName addindicator ?*arg...*?

pathName addtoolbar

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName getindicator *index*

pathName getmenu *menuid*

pathName gettoolbar *index*

pathName setmenustate *tag state*

pathName showstatusbar *name*

pathName showtoolbar *index bool*

DESCRIPTION

MainFrame manage toplevel to have:

- simple menu creation, with automatic accelerator bindings and DynamicHelp association,
- one or more toolbars that user can hide,
- a status bar, displaying a user message or a menu description, and optionally a ProgressBar.

WIDGET-SPECIFIC OPTIONS

-height

Specifies the desired height for the user frame in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero (the default) then the window will not request any size at all.

-menu (read-only)

This option describes the menu. This is a list whose each five elements describe one cascade menu. It has the following form: {*menuname tags menuId tearoff menuentries*...} where *menuentries* is a list where each element describe one menu entry, which can be:
- for a separator: {**separator**}
- for a command: {**command** *menuname* ?*tags*? ?*description*? ?*accelerator*? ?*option value*? ...}

- for a check button: {**checkbutton** *menuname* ?*tags*? ?*description*? ?*accelerator*? ?*option value*? ...}
- for a radio button: {**radiobutton** *menuname* ?*tags*? ?*description*? ?*accelerator* ?*option value*? ...}
- for a cascade menu: {**cascade** *menuname tags menuId tearoff menuentries*}

where:
- *menuname* is the name of the menu. If it contains a &, the following character is automatically converted to the corresponding **-underline** option of **menu add** command.
- *tags* is the tags list for the entry, used for enabling or disabling menu entries with **MainFrame::setmenustate**.
- *menuId* is an id for the menu, from which you can get menu pathname with **MainFrame::getmenu**.
- *tearoff* specifies if menu has tearoff entry.
- *description* specifies a string for DynamicHelp.
- *accelerator* specifies a key sequence. It is a list of two elements, where the first is one of **Ctrl**, **Alt** or **CtrlAlt**, and the second as letter or a digit. An accelerator string is build and corresponding binding set on the toplevel to invoke the menu entry.
- *option value* specifies additionnal options for the entry (see **menu add** command).

Each value enclosed by ? are optional and defaulted to empty string, but must be provided if one or more following options is not empty. Example:

```
set descmenu {
    "&File" {} {} 0 {
        {command "&New"     {} "Create a new document"     {Ctrl n} -command
    Menu::new}
        {command "&Open..." {} "Open an existing document" {Ctrl o} -command
    Menu::open}
        {command "&Save"    open "Save the document" {Ctrl s} -command
    Menu::save}
        {cascade  "&Export"  {} export 0 {
            {command "Format &1" open "Export document to format 1" {} -command
    {Menu::export 1}}
            {command "Format &2" open "Export document to format 2" {} -command
    {Menu::export 2}}
        }}
        {separator}
        {cascade "&Recent files" {} recent 0 {}}
        {separator}
        {command "E&xit" {} "Exit the application" {} -command Menu::exit}
    }
    "&Options" {} {} 0 {
        {checkbutton "Toolbar" {} "Show/hide toolbar" {}
            -variable Menu::_drawtoolbar
            -command  {$Menu::_mainframe showtoolbar toolbar $Menu::_drawtoolbar}
        }
    }
}
```

-separator (read-only)

Specifies if separator should be drawn at the top and/or at the bottom of the user window. Must be one of the values **none**, **top**, **bottom** or **both**. It depends on the relief of subwidgets of user window.

-textvariable
Specifies the textvariable option for the label of the status bar. DynamicHelp description of menu entries are mapped to this variable at the creation of the MainFrame. If this variable is changed by MainFrame::configure, menu description will not be available. You change the text of the label by modifying the value of the variable.

-width
Specifies the desired width for the user frame in any of the forms acceptable to Tk_GetPixels. If this option is less than or equal to zero (the default) then the window will not request any size at all.

WIDGET COMMAND

***pathName* addindicator ?*arg...*?**
Add an indicator box at the right of the status bar. Each indicator are added from left to right. An indicator is a Tk label widget configured with option-value pair given by ?*arg...*?. **-relief** and **-borderwidth** options are respetively defaulted to *sunken* and 1. Returns the pathname of the created label.

***pathName* addtoolbar**
Add a toolbar to the MainFrame. Returns the pathname of the new window where to place toolbar items.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Returns the pathname of the user window.

pathName* getindicator *index
Returns the pathname of the *index*th added indicator.

pathName* getmenu *menuid
Returns the pathname of the menu whose id is *menuid*.

pathName* getmenustate *tag state
Returns the state of the given menu *tag*.

pathName* gettoolbar *index

Returns the pathname of the *index*th added toolbar.

pathName* setmenustate *tag state
Set the **-state** option value of all the menu entries that have the tag *tag* to *state*. A menu entry is disabled, if one of its associated tags have state **disabled**.

pathName* showstatusbar *name
name is one of **none**, **status** or **progression**. Use **none** to hide the status bar, **status** to display the label only, or **progression** to display the label and the ProgressBar.

pathName* showtoolbar *index bool
Hide if *bool* is 0, or show if *bool* is 1 the *index*th added toolbar. To prevent your toplevel from resizing while hiding/showing toolbar, do [wm geometry $top [wm geometry $top]] when it is managed.

MessageDlg

Keywords: MessageDlg

NAME
MessageDlg - Message dialog box

CREATION
MessageDlg *pathName* ?*option value...*?

STANDARD OPTIONS

-anchor	-font
-foreground or -fg	-padx
-pady	

OPTIONS from Dialog

-background or -bg	-cancel
-default	-parent

WIDGET-SPECIFIC OPTIONS

-aspect	-buttons
-buttonwidth	-icon
-justify	-message
-title	-type
-width	

DESCRIPTION

MessageDlg provides a simple way to display a message dialog. MessageDlg::**create** creates the message dialog, displays it and return the index of the pressed button, or -1 if it is destroyed. When returning, the dialog no longer exists.

WIDGET-SPECIFIC OPTIONS

-aspect
Specifies a non-negative integer value indicating desired aspect ratio for the text. The aspect ratio is specified as 100*width/height. 100 means the text should be as wide as it is tall, 200 means the text should be twice as wide as it is tall, 50 means the text should be twice as tall as it is wide, and so on. Used to choose line length for text if **width** option isn't specified. Defaults to 150.

-buttons
Specifies a list of buttons to display when **type** option is *user*. If a button has a symbolic name, its associated text will be displayed.

-buttonwidth
Specifies the standard width of the buttons in the dialog.

-icon
Specifies an icon to display. Must be one of the following: **error**, **info**, **question** or **warning**.

-justify
Specifies how to justify lines of text. Must be one of **left**, **center**, or **right**. Defaults to **left**. This option works together with the **anchor**, **aspect**, **padx**, **pady**, and **width** options to provide a variety of arrangements of the text within the window. The **aspect** and **width** options determine the amount of screen space needed to display the text. The **anchor**, **padx**, and **pady** options determine where this rectangular area is displayed within the widget's window, and the **justify** option determines how each line is displayed within that rectangular region. For example, suppose **anchor** is **e** and **justify** is **left**, and that the message window is much larger than needed for the text. The the text will displayed so that the left edges of all the lines line up and the right edge of the longest line is **padx** from the right side of the window; the entire text block will be centered in the vertical span of the window.

-message
Specifies the message to display in this message box.

-title
Specifies a string to display as the title of the message box. If the value is empty (the default), a default title will be set corresponding to the **icon** option. The default associated title is in english, and can be modified to set it in another language by specifying the resource:

```
    *MessageDlg.nameTitle:   value
```

or the equivalent tcl command:

```
    option add *MessageDlg.nameTitle value
```

where *name* is the name of an icon as defined in the **icon** option. For example, for french language, you can specify for a warning dialog:

```
option add *MessageDlg.warningTitle  "Attention"
```

-type
Specifies a set of buttons to be displayed. The following values are possible:
abortretryignore
Displays three buttons whose symbolic names are **abort**, **retry** and **ignore**.

ok

Displays one button whose symbolic name is **ok**.

okcancel

Displays two buttons whose symbolic names are **ok** and **cancel**.

retrycancel

Displays two buttons whose symbolic names are **retry** and **cancel**.

yesno

Displays two buttons whose symbolic names are **yes** and **no**.

yesnocancel

Displays three buttons whose symbolic names are **yes**, **no** and **cancel**.

user

Displays buttons of **-buttons** option.

-width
Specifies the length of lines in the window. If this option has a value greater than zero then the **aspect** option is ignored and the **width** option determines the line length. If this option has a value less than or equal to zero, then the **aspect** option determines the line length.

NoteBook

Keywords: NoteBook

NAME
NoteBook - Notebook manager widget

CREATION
NoteBook *pathName* ?*option value...*?

STANDARD OPTIONS

-font

OPTIONS from ArrowButton

-activebackground	-activeforeground
-background or -bg	-borderwidth or -bd
-disabledforeground	-foreground or -fg
-repeatdelay	-repeatinterval

WIDGET-SPECIFIC OPTIONS

-arcradius	-height
-homogeneous	-side
-tabbevelsize	-tabpady
-width	

WIDGET COMMAND

pathName bindtabs *event script*

pathName cget *option*

pathName compute_size

pathName configure ?*option*? ?*value option value* ...?

pathName delete *page* ?*destroyframe*?

pathName getframe *page*

pathName index *page*

pathName insert *index page* ?*option value...*?

pathName itemcget *page option*

pathName itemconfigure *page* ?*option*? ?*value option value* ...?

pathName move *page index*

pathName page *first* ?*last*?

pathName pages ?*first*? ?*last*?

pathName raise ?*page*?

pathName see *page*

DESCRIPTION

NoteBook widget manage a set of pages and displays one of them.

WIDGET-SPECIFIC OPTIONS

-arcradius
Specifies the amount of rounding effect at the corners of a tab. This value can be adjusted from 0 to 8 pixels with the default being 2 pixels. Usually, small values are preferable.

-height
Specifies the desired height for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call NoteBook:: **compute_size** to make NoteBook larger enough to contains the largest page.

-homogeneous
Specifies wether or not the label of the pages must have the same width.

-side
Specifies the side where to place the label of the pages. Must be one of **top** or **bottom**.

-tabbevelsize
Specifies the amount of bevel the tabs should have. This value can be adjusted from 0 to 8 pixels with the default being 0 pixels. A zero pixel bevel is essentially rectangular while non-zero bevel size will look trapezoidal.

-tabpady
Specifies the padding between the text in the tab and the top and bottom of the tab. Padding may be a list of two values to specify padding for top and bottom separately. Padding defaults to {0 6}.

-width
Specifies the desired width for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call NoteBook:: **compute_size** to make NoteBook larger enough to contains the largest page.

WIDGET COMMAND

pathName* bindtabs *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on a tab. The page identifier on which the event occurs is appended to the command.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* compute_size**
This command can be called to make the NoteBook large enough to contain the largest page. Note that if all pages use -createcmd, they will have no requested size.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete *page* ?*destroyframe*?**
Deletes the page *page*. If *destroyframe* is 1 (the default), the frame associated to *page* is destroyed. If *destroyframe* is 0, the frame is not destroyed and is reused by further call to **insert** with the same *page*.

pathName* getframe *page
Returns the pathname of the page *page*.

pathName* index *page
Return the numerical index corresponding to the item.

***pathName* insert *index page* ?*option value...*?**
Insert a new page idendified by *page* at position *index* in the pages list. *index* must be numeric or **end**. The pathname of the new page is returned.

-createcmd
Specifies a command to be called the first time the page is raised.

-image
Specifies an image to display for the page at the left of the label

-leavecmd
Specifies a command to be called when a page is about to be leaved. The command must return 0 if the page can not be leaved, or 1 if it can.

-raisecmd
Specifies a command to be called each time the page is raised.

-state
Specifies the state of the page. Must be **normal** or **disabled**.

-text
Specifies a label to display for the page.

pathName* itemcget *page option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *page* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If

options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* move *page index
Moves *page* tab to index *index*.

***pathName* page *first* ?*last*?**
Its use is deprecated. Use pages instead. If *last* is omitted, returns the page at index *first* , or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* pages ?*first*? ?*last*?**
If *first* and *last* are omitted, returns the list of all pages. If *first* is specified and *last* omitted, returns the page at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* raise ?*page*?**
Raise the page *page*, or return the raised page if *page* is omitted.

pathName* see *page
Scrolls labels to make the label of the page *page* visible.

PagesManager

Keywords: PagesManager

NAME
PagesManager - Pages manager widget

CREATION
PagesManager *pathName* ?*option value...*?

STANDARD OPTIONS
-background

WIDGET-SPECIFIC OPTIONS
-height

WIDGET COMMAND
pathName add *page*

pathName cget *option*

pathName compute_size

pathName configure ?*option*? ?*value option value* ...?

pathName delete *page*

pathName getframe *page*

pathName page *first* ?*last*?

pathName pages ?*first*? ?*last*?

pathName raise ?*page*?

DESCRIPTION

PagesManager widget manages a set of pages and displays one of them. PagesManager does not provide any user access method, as NoteBook does, so it can be done through a listbox, a menu, radiobutton, or whatever. The widget shows no pages during creation; there must be an explicit call to **raise** to display one.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call PagesManager:: **compute_size** to make PagesManager larger enough to contains the largest page.

-width
Specifies the desired width for the pages. If this option is equal to zero (the default) then the window will not request any size at all. In this case, user may want to call PagesManager:: **compute_size** to make PagesManager larger enough to contains the largest page.

WIDGET COMMAND

pathName* add *page
Add a new page identified by *page*, which is an arbitrary identifier. The pathname of the new page is returned and widgets for the page should be created with this as the parent or ancestor. However, PagesManager manages its own geometry. **pack**, **grid** or an equivalent should not be used with the pathname returned by **add**.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* compute_size**
This command can be called to make the PagesManager large enough to contain the largest page.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have

any of the values accepted by the creation command. Read-only options are not be modified.

pathName* delete *page
Deletes the page *page*.

pathName* getframe *page
Returns the pathname of the page *page*.

***pathName* page *first* ?*last*?**
Its use is deprecated. Use pages instead. If *last* is omitted, returns the page at index *first* , or an empty string if *first* refers to a non-existent element. If *last* is specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* pages ?*first*? ?*last*?**
If *first* and *last* are omitted, returns the list of all pages. If *first* is specified and *last* omitted, returns the page at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the pages between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* raise ?*page*?**
Raise the page *page*, or return the raised page if *page* is omitted.

PanedWindow

Keywords: PanedWindow

NAME
PanedWindow - Tiled layout manager widget

CREATION
PanedWindow *pathName* ?*option value...*?

STANDARD OPTIONS
-background or -bg

WIDGET-SPECIFIC OPTIONS

-activator	-pad
-side	-weights
-width	

WIDGET COMMAND
pathName add ?*option value...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value ...*?

pathName getframe *index*

DESCRIPTION

PanedWindow is a widget that lays out children in a vertically or horizontally tiled format. The user can adjust the size of the panes, with a pane control sash created between children.

WIDGET-SPECIFIC OPTIONS

-activator (read-only)
By default, the control sash is either a button or a line depending upon the underlying operating system. This sash may be explicitly set using this option. If set to **line** then the sash consists of just a dividing line. Otherwise if set to **button** then it constists of a knob that can be dragged.

-pad (read-only)
Specifies additional space between the button of the sash and children.

-side (read-only)
Specifies the side of the sash, which implies the layout: **top** or **bottom** (horizontal layout), **left** or **right** (vertical layout).

-weights (read-only)
Specifies how the weights specified when adding panes should be used. Must be **extra** or **available**. When using *extra*, only extra space is devided among the diffferent panes relative to their weight. When using *available*, all space is devided among the diffferent panes relative to their weight. Default value for **weights** is *extra*.

-width (read-only)
Specifies the width of the button of the sash. This option is ignored if the activator is set to **line**.

WIDGET COMMAND

***pathName* add ?*option value...*?**
This command add a new pane. The new pane is placed below the previous pane for vertical layout or at right for horizontal layout. This command returns a frame where user can place its widget. Valid options are:

-minsize
Specifies the minimum size requested for the pane. See the **grid** command for more information.

-weight
Specifies the relative weight for apportioning any extra spaces among panes. See the **grid** command for more information.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

pathName* getframe *index

Returns the pathname of the *index*th added pane.

PanelFrame

Keywords: PanelFrame

NAME

PanelFrame - Frame with a boxed title area

CREATION

PanelFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-text	
-textvariable	
-font	-relief

WIDGET-SPECIFIC OPTIONS

-height	
-width	
-panelbackground	-panelforeground
-ipad	

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

PanelFrame creates a frame area with a boxed title area. The boxed title area contains a label ands allows for other items to be placed in it. This can serve like a mini-toolbar.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the widget.

-ipad
The spacing to place around individual panel area items.

-panelbackground
The color for the panel area background. Defaults to the selection highlight background color.

-panelforeground
The color for the title text. Defaults to the selection highlight foreground color.

-width
Specifies the desired width for the widget.

WIDGET COMMAND

***pathName* add widget ?*option value option value ...*?**
Add a widget to the panel. Widgets are **pack**ed in. Possible options are:

-side
Side to place item on (defaults to **right**).

-fill
Whether to fill space (defaults to **none**).

-expand
Whether to expand space (defaults to **0**).

-pad
Override of the widget's **-ipad** option for this item.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete widget ?*widget ...*?**
Delete a widget and associated state from the panel.

***pathName* getframe**
Get the frame widget for the status bar in which status bar items should be created.

***pathName* items**
List of items in the status bar.

PasswdDlg

Keywords: PasswdDlg

NAME

PasswdDlg - Login/Password dialog box

CREATION

PasswdDlg *pathName* ?*option value...*?

OPTIONS from Dialog

-anchor	-background or -bg
-homogeneous	-modal
-padx	-pady
-parent	-spacing
-title	

OPTIONS from LabelEntry

-background or -bg	-borderwidth or -bd
-disabledforeground	-entrybg
-entryfg	-exportselection
-font	-foreground or -fg
-helptype	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-labelanchor
-labelfont	-labelheight
-labeljustify	-labelwidth
-loginhelptext (see -helptext)	-loginhelpvar (see -helpvar)
-loginlabel (see -label)	-logintext (see -text)
-logintextvariable (see -textvariable)	-loginunderline (see -underline)
-passwdeditable (see -editable)	-passwdhelptext (see -helptext)
-passwdhelpvar (see -helpvar)	-passwdlabel (see -label)

-passwdstate (see -state)	-passwdtext (see -text)
-passwdtextvariable (see -textvariable)	-passwdunderline (see -underline)
-relief	-selectbackground
-selectborderwidth	-selectforeground
-wraplength	

WIDGET-SPECIFIC OPTIONS

-command
-type

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

PasswdDlg provides a simple way to display a login/password dialog. PasswdDlg::**create** creates the dialog, displays it, and return the value of login and password in a list, or an empty list if it is destroyed or user press cancel. When returning, the dialog no longer exists.
Additionnal resources can be set to modify other text:

```
    *loginName     Label for login LabelEntry
    *passwordName  Label for password LabelEntry
```

WIDGET-SPECIFIC OPTIONS

-command

Specifies a command to call when user press ok button.

-type

Specifies a set of buttons to be displayed. The following values are possible:

ok

Displays one button whose symbolic name is **ok**.

okcancel

Displays two buttons whose symbolic names are **ok** and **cancel**.

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

Stephane Lavirotte

(Stephane.Lavirotte@sophia.inria.fr)

ProgressBar

Keywords: ProgressBar

NAME

ProgressBar - Progress indicator widget

CREATION

ProgressBar *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-foreground or -fg	-orient
-relief	
-troughcolor	

WIDGET-SPECIFIC OPTIONS

-height
-maximum
-type
-variable
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

ProgressBar widget indicates the user the progress of a lengthly operation. It is used by MainFrame and ProgressDlg.

WIDGET-SPECIFIC OPTIONS

-height
Specifies the desired height for the progress indicator.

-maximum
Specifies the maximum value of the variable. This value must be greater than zero.

-type
Specifies the type of the ProgressBar. Must be one of **normal**, **incremental**, **infinite** or **nonincremental_infinite**.

If **type** is *normal*, the progress indicator is drawn proportional to the variable value and **maximum** option each time the variable is set.

If **type** is *incremental*, the value of the progress indicator is maintained internally, and incremented each time the variable is set by its value. The progress indicator is drawn proportional to the internal value and **maximum** option.

If **type** is *infinite*, the value of the progress indicator is maintained internally, and incremented each time the variable is set by its value. The progress indicator moves from left to right if internal value (modulo **maximum**) is less than **maximum**/2, and from right to left if internal value is greater than **maximum**/2.

If **type** is *nonincremental_infinite*, the value of the progress indicator taken from the variable value, The progress indicator moves from left to right if variable value (modulo **maximum**) is less than **maximum**/2, and from right to left if internal value is greater than **maximum**/2.

See **-variable** option for special case of its value,

Default value for **type** is *normal*.

-variable
Specifies the variable attached to the progress indicator. Progress indicator is updated when the value of the variable changes. If the value of the variable is negative, the progress indicator is not displayed (it is drawn flat with **background** color - usefull for ProgressDlg to make it invisible). If its value 0, progress indicator is reinitialized.

-width
Specifies the desired width for the progress indicator.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

ProgressDlg

Keywords: ProgressDlg

NAME

ProgressDlg - Progress indicator dialog box

CREATION

ProgressDlg *pathName* ?*option value...*?

STANDARD OPTIONS

-font
-textvariable

OPTIONS from Dialog

-background or -bg	-parent
-separator	-title

OPTIONS from ProgressBar

-background or -bg	-borderwidth or -bd
-foreground or -fg	-maximum
-relief	-troughcolor
-type	-variable

WIDGET-SPECIFIC OPTIONS

-command
-height
-stop
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

ProgressDlg provides a simple way to display a progress indicator dialog. ProgressDlg::**create** creates the dialog, displays it, set a local grab to it and immediatly return. The dialog is updated by modifying the value of the variable of options **-textvariable** and **-variable**. You have to destroy the dialog after use.

WIDGET-SPECIFIC OPTIONS

-command
Specifies a command to call when user press stop button. Note that it is the program's responsibility to periodically call **update** so that button press events can be generated.

-height
Specifies a desired height for the label in lines of text.

-stop
Specifies the text of the button typically used to stop process. If empty, no button will be drawn. This can be a symbolic name.

-width
Specifies a desired width for the label in characters.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

ScrollView

Keywords: ScrollView

NAME
ScrollView - Display the visible area of a scrolled window

CREATION

ScrollView *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-cursor	

WIDGET-SPECIFIC OPTIONS

-fill
-height

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

ScrollView displays the visible area of a scrolled window within its scroll region.

WIDGET-SPECIFIC OPTIONS

-fill
Specifies the fill color of the rectangle.

-foreground
Specifies the color of the border of the rectangle.

-height
Specifies the desired height for the ScrollView.

-width
Specifies the desired width for the ScrollView.

-window
Specifies the window to view. This widget must have **-xscrollcommand** and **-yscrollcommand** options, and respond to **xview** and **yview** command. In order to make ScrollView working with other scrollbar, **-xscrollcommand** and **-yscrollcommand** options of the widget must be set before the widget is passed to the **-window** option of the ScrollView (for example, if the widget is handled by a ScrolledWindow, call **setwidget** before setting **-window** option).

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

BINDINGS

ScrollableFrame

Keywords: ScrollableFrame

NAME

ScrollableFrame - Scrollable frame containing widget

CREATION

ScrollableFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-xscrollcommand
-yscrollcommand	

WIDGET-SPECIFIC OPTIONS

-areaheight
-areawidth
-constrainedheight
-constrainedwidth
-height
-width
-xscrollincrement
-yscrollincrement

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName see *widget* ?*vert*? ?*horz*?

pathName xview ?*arg...*?

pathName yview ?*arg...*?

DESCRIPTION

ScrollableFrame widget containing widget.

WIDGET-SPECIFIC OPTIONS

-areaheight
Specifies the height for the scrollable area. If zero, then the height of the scrollable area is made just large enough to hold all its children.

-areawidth
Specifies the width for the scrollable area. If zero, then the width of the scrollable area window is made just large enough to hold all its children.

-constrainedheight
Specifies whether or not the scrollable area should have the same height of the scrolled window. If true, vertical scrollbar is not needed.

-constrainedwidth
Specifies whether or not the scrollable area should have the same width of the scrolled window. If true, horizontal scrollbar is not needed.

-height
Specifies the desired height for the window in pixels.

-width
Specifies the desired width for the window in pixels.

-xscrollincrement
See **xscrollincrement** option of **canvas** widget.

-yscrollincrement
See **yscrollincrement** option of **canvas** widget.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more

option-value pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Return the pathname of the scrolled frame where widget should be created.

***pathName* see *widget* ?*vert*? ?*horz*?**
Arrange scrollable area to make *widget* visible in the window. *vert* and *horz* specify which part of *widget* must be preferably visible, in case where *widget* is too tall or too large to be entirely visible. *vert* must be **top** (the default) or **bottom**, and *horz* must be **left** (the default) or **right**. If *vert* or *horz* is not a valid value, area is not scrolled in this direction.

***pathName* xview ?*arg...*?**
Standard command to enable horizontal scrolling of *pathName*.

***pathName* yview ?*arg...*?**
Standard command to enable vertical scrolling of *pathName*.

ScrolledWindow

Keywords: ScrolledWindow

NAME
ScrolledWindow - Generic scrolled widget

CREATION
ScrolledWindow *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-relief	

WIDGET-SPECIFIC OPTIONS

-auto
-ipad
-managed
-scrollbar
-sides
-size

WIDGET COMMAND
pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

pathName setwidget *widget*

DESCRIPTION

ScrolledWindow enables user to create easily a widget with its scrollbar. Scrollbars are created by ScrolledWindow and scroll commands are automatically associated to a scrollable widget with **ScrolledWindow::setwidget**.

WIDGET-SPECIFIC OPTIONS

-auto
Specifies the desired auto managed scrollbar: none means scrollbar are always drawn horizontal means horizontal scrollbar is drawn as needed vertical means vertical scrollbar is drawn as needed both means horizontal and vertical scrollbars are drawn as needed (default value)

-ipad (read-only)
Padding in pixels between client widget and scrollbars. Default value: **1**.

-managed (read-only)
If true, scrollbar are managed during creation, so their size are included in the requested size of the ScrolledWindow. If false, they are not. Default value: **true**.

-scrollbar
Specifies the desired scrollbar: **none**, **horizontal**, **vertical** or **both** (default value).

-sides (read-only)
Side of the scrollbars. Possible values are: **ne**, **en**, **nw**, **wn**, **se** (default value), **es**, **sw**, **ws**.

-size (read-only)
Size of the scrollbars in pixels. Use 0 for standard size (default value).

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**

Return the pathname of the frame where the scrolled widget should be created. This command is no longer needed. You can directly create the scrolled widget as the child of *pathName*.

pathName* setwidget *widget
Associate *widget* to the the scrollbars. *widget* becomes managed by the ScrolledWindow. The user should not attempt to manage *widget* until it is no longer managed by the ScrolledWindow. *widget* must be a scrollable widget, i.e. have the options **xscrollcommand**/**yscrollcommand** and the command **xview**/**yview**, such as canvas or text.

SelectColor

Keywords: SelectColor

NAME
SelectColor - Color selection widget

CREATION
SelectColor *pathName* ?*option value...*?

WIDGET-SPECIFIC OPTIONS

-color	-parent
-placement	-title
-type	

WIDGET COMMAND
pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

SelectColor::dialog *pathName?option value ...?*

SelectColor::menu *pathName placement?option value ...?*

SelectColor::setcolor *index color*

DESCRIPTION

SelectColor provides a simple way to select color. It can be displayed as a dialog box or as a menubutton.

WIDGET-SPECIFIC OPTIONS

-color
Specifies the color value of the widget.

-parent
Parent of the Dialog. Dialog is centered in its parent. If empty, it is centered in root window.

-placement
Where to place the *popup* color dialog when displaying it. Must be any of: **at**, **center**, **left**, **right**, **above**, or **below**. If *-parent* is specified, placement will be in relation to the parent widget.

-title
Title of the Dialog toplevel.

-type (read-only)
Specifies the type of the SelectColor widget. Must be **dialog** or **popup**. If **type** option is *dialog*, SelectColor::**create** directly creates the dialog, displays it and return an empty string if cancel button is pressed or if dialog is destroyed, and the selected color if ok button is pressed. In all cases, dialog is destroyed. If **type** option is *popup*, SelectColor::**create** creates a small, popup dialog with a small set of predefined colors and a button to activate a full color dialog.

WIDGET COMMAND

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

SelectColor::dialog *pathName?option value ...?*
Creates a dialog for the user to select a custom color.

SelectColor::menu *pathName placement?option value ...?*
Creates a small, popup dialog for the user to select from a predefined list of colors with an additional button to display a full color dialog.

placement can be any of **at**, **center**, **left**, **right**, **above**, or **below**. If *-parent* is specified, placement will be in relation to the parent widget.

SelectColor::setcolor *index color*
Set the value of user predefined color at index *index* to *color*. *index* must be between 0 and 10.

SelectFont

Keywords: SelectFont

NAME
SelectFont - Font selection widget

CREATION

SelectFont *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-font

WIDGET-SPECIFIC OPTIONS

-command

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

SelectFont::loadfont

DESCRIPTION

SelectFont provides a simple way to choose font. It can be displayed as a dialog box or as a toolbar.
Textual items in Dialog box uses **-name** options so they can be translated to any language. Symbolic name used are **ok**, **cancel**, **font**, **size**, **style**, **bold**, **italic**, **underline** and **overstrike**.

WIDGET-SPECIFIC OPTIONS

-command
Specifies a command to call when user select a new font when SelectFont **type** option is *toolbar*.

-initialcolor
If specified, add an additional button that lets the user pick a color. This option is ignored if **type** is *toolbar*.

-nosizes
If true, don't show the listbox containing valid font sizes. This option is ignored if **type** is *toolbar*.

-parent
Parent of the Dialog. Dialog is centered in its parent. If empty, it is centered in root window.

-sampletext
Specifies the text displayed in the preview area.

-title
Title of the Dialog toplevel.

-type
Specifies the type of the SelectFont widget. Must be **dialog** or **toolbar**. If **type** option is *dialog*, SelectFont::**create** directly creates the dialog, displays it and return an empty string if cancel button is pressed or if dialog is destroyed, and the selected font (and color if

initialcolor) if ok button is pressed. In all cases, dialog is destroyed. If **type** option is *toolbar*, SelectFont::**create** returns the pathname of the widget created.

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

SelectFont::loadfont

Load the font available in the system.

Separator

Keywords: Separator

NAME

Separator - 3D separator widget

CREATION

Separator *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-orient

WIDGET-SPECIFIC OPTIONS

-relief

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

Separator is a widget that display an horizontal or vertical 3-D line.

WIDGET-SPECIFIC OPTIONS

-relief

Specifies the relief of the Separator. Must be **groove** (the default) or **ridge**.

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

SpinBox

Keywords: SpinBox

NAME

SpinBox - SpinBox widget

CREATION

SpinBox *pathName* ?*option value...*?

OPTIONS from ArrowButton

-background or -bg	-disabledforeground
-foreground or -fg	-repeatdelay
-repeatinterval	-state

OPTIONS from Entry

-command	-disabledforeground
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-droptypes	-editable
-entrybg (see -background)	-entryfg (see -foreground)
-exportselection	-font
-helptext	-helptype

-helpvar	-highlightbackground
-highlightcolor	-highlightthickness
-insertbackground	-insertborderwidth
-insertofftime	-insertontime
-insertwidth	-justify
-selectbackground	-selectborderwidth
-selectforeground	-show
-state	-takefocus
-text	-textvariable
-width	-xscrollcommand

WIDGET-SPECIFIC OPTIONS

-modifycmd
-range
-values

WIDGET COMMAND

pathName bind ?*arg...*?

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getvalue

pathName setvalue *index*

DESCRIPTION

SpinBox widget enables the user to select a value among a list given by the **values** option or a set of values defined by a mininum, a maximum and an increment. Notice that **range** option defines a list of values, so **getvalue** and **setvalue** work with both values and range.

WIDGET-SPECIFIC OPTIONS

-modifycmd

Specifies a Tcl command called when the user modify the value of the SpinBox.

-range
Specifies a list of three intergers (or real) describing the minimum, maximum and increment of the SpinBox.

-values
Specifies the values accepted by the SpinBox. This option takes precedence over **range** option.

WIDGET COMMAND

***pathName* bind ?*arg...*?**
Set bindings on the entry widget.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getvalue**
Returns the index of the current text of the SpinBox in the list of values, or -1 if it doesn't match any value.

pathName* setvalue *index
Set the text of the SpinBox to the value indicated by *index* in the list of values. *index* may be specified in any of the following forms:

last
Specifies the last element of the list of values.

first
Specifies the first element of the list of values.

next
Specifies the element following the current (ie returned by **getvalue**) in the list of values.

previous
Specifies the element preceding the current (ie returned by **getvalue**) in the list of values.

@*number*
Specifies the integer index in the list of values.

BINDINGS

When Entry of the SpinBox has the input focus, it has the following bindings, in addition to the default Entry bindings:

- Page up set the value of the SpinBox to the last value.

- Page down set the value of the SpinBox to the first value.
- Arrow up set the value of the SpinBox to the next value.
- Arrow down set the value of the SpinBox to the previous value.

StatusBar

Keywords: StatusBar

NAME

StatusBar - status bar widget

CREATION

StatusBar *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-foreground or -fg	-orient
-relief	
-troughcolor	

WIDGET-SPECIFIC OPTIONS

-height
-showresize
-ipad
-pad
-width

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

DESCRIPTION

StatusBar widget is a simple container widget with a corner resize control, meant to be placed at the bottom of a toplevel dialog.

WIDGET-SPECIFIC OPTIONS

-height

Specifies the desired height for the widget.

-showresize

Specifies whether to show the corner resize control.

-pad
The spacing to place around the status bar.

-ipad
The spacing to place around individual status bar items.

-width
Specifies the desired width for the widget.

WIDGET COMMAND

***pathName* add widget ?*option value option value ...*?**
Add a widget to the status bar. Possible options are:

-weight
Weighting of this item for resizing (passed to **grid**).

-separator
Whether to use a separator for this item.

-sticky
Passed on to grid.

-pad
Override of the widget's **-ipad** option for this item.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete widget ?*widget ...*?**
Delete a widget and associated state from the status bar.

***pathName* getframe**
Get the frame widget for the status bar in which status bar items should be created.

***pathName* items**
List of items in the status bar.

TitleFrame

Keywords: TitleFrame

NAME
TitleFrame - Frame with a title

CREATION

TitleFrame *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-font	
-relief	

WIDGET-SPECIFIC OPTIONS

-baseline

WIDGET COMMAND

pathName cget *option*

pathName configure ?*option*? ?*value option value* ...?

pathName getframe

DESCRIPTION

TitleFrame enables user to create a frame with a title like XmFrame Motif widget.

WIDGET-SPECIFIC OPTIONS

-baseline

Specifies the vertical alignment of the title: **top**, **center** or **bottom**.

-ipad

Specifies a pad between the border of the frame and the user frame. The value is in screen units.

-side

Specifies the horizontal alignment of the title: **left**, **center** or **right**.

WIDGET COMMAND

pathName* cget *option

Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* configure ?*option*? ?*value option value* ...?**

Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* getframe**
Return the frame where the user can create any other widget.

Tree

Keywords: Tree

NAME
Tree - Tree widget

CREATION
Tree *pathName* ?*option value...*?

STANDARD OPTIONS

-background or -bg	-borderwidth or -bd
-cursor	-highlightbackground
-highlightcolor	-highlightthickness
-relief	-selectbackground
-selectforeground	-takefocus
-xscrollcommand	-yscrollcommand

WIDGET-SPECIFIC OPTIONS

-closecmd	-crossfill
-crossclosebitmap	-crosscloseimage
-crossopenbitmap	-crossopenimage
-deltax	-deltay
-dragenabled	-dragendcmd
-dragevent	-draginitcmd
-dragtype	-dropcmd
-dropenabled	-dropovercmd
-dropovermode	-droptypes
-height	-linesfill
-linestipple	-opencmd
-padx	-redraw
-selectcommand	-selectfill

-showlines	-width

WIDGET COMMAND

pathName bindArea *event script*

pathName bindImage *event script*

pathName bindText *event script*

pathName cget *option*

pathName closetree *node*

pathName configure ?*option*? ?*value option value* ...?

pathName delete ?*arg...*?

pathName edit *node text* ?*verifycmd*? ?*clickres*? ?*select*?

pathName exists *node*

pathName find *findinfo* ?*confine*?

pathName index *node*

pathName insert *index parent node* ?*option value...*?

pathName itemcget *node option*

pathName itemconfigure *node* ?*option*? ?*value option value* ...?

pathName line *node*

pathName move *parent node index*

pathName nodes *node* ?*first*? ?*last*?

pathName opentree *node*

pathName parent *node*

pathName reorder *node neworder*

pathName see *node*

pathName selection *cmd* ?*arg...*?

pathName toggle *node*

pathName visible *node*

pathName xview ?*arg...*?

pathName yview ?*arg...*?

DESCRIPTION

Tree widget uses canvas to display a hierarchical list of items (called nodes). Each node is composed of a label with its own font and foreground attributes, and an optional image or window. Each node can have a list of subnodes, which can be collapsed or expanded. Each node is drawn in a single line, whose height is defined by the **deltay** option, so they must have at most this height. A node is uniquely identified by a string given at creation (by the **insert** command). The node named *root* is the root of the tree and is not drawn. The tree structure is directly maintained by the widget.

WIDGET-SPECIFIC OPTIONS

-closecmd
Specifies a command to be called when user close a node. The closed node is appended to the command.

-crossfill
Specifies a foreground color for the cross bitmap.

-crossclosebitmap
Specifies a bitmap to be displayed in place of the standard cross when a node is closed.

-crosscloseimage
Specifies an image to be displayed in place of the standard cross when a node is closed. Overrides the -crossclosebitmap option.

-crossopenbitmap
Specifies a bitmap to be displayed in place of the standard cross when a node is open.

-crossopenimage
Specifies an image to be displayed in place of the standard cross when a node is open. Overrides the -crossopenbitmap option.

-deltax
Specifies horizontal indentation between a node and its children.

-deltay
Specifies vertical size of the nodes.

-dragenabled
A boolean specifying if drag is enabled.

-dragendcmd
Specifies a command to be called when drag ended. **dragendcmd** must be a command conforming to the description of the option **dragendcmd** of **DragSite::register**.

-dragevent
Specifies the number of the mouse button associated to the drag. Must be **1**, **2** or **3**.

-draginitcmd
Tree has a command wrapper for *drag-init* events. This command refused the drag if no node is designated. In other cases: If **draginitcmd** is empty, it returns:
- the value of option **dragtype** or *TREE_NODE* if empty as the data type,
- *{copy move link}* as the operations,
- the node identifier as the data.

If **draginitcmd** is not empty, it is called with the following arguments:
- the pathname of the tree,
- the identifier of the dragged node,
- the toplevel created to represent dragged data.

and must return a value conforming to **draginitcmd** option described in **DragSite::register**.

-dragtype
Specifies an alternate type of dragged object.

-dropcmd
Tree has a command wrapper for *drop* events. This command stops auto scrolling and extract node and position. If **dropcmd** is not empty, it is called with the following arguments:
- the pathname of the tree,
- the pathname of the drag source,
- a list describing where the drop occurs. It can be:
- the current operation,
- the data type,
- the data.
  - *{***widget***}*,
  - *{***node** *node}* or
  - *{***position** *node index}*.

-dropenabled
A boolean specifying if drop is enabled.

-dropovercmd
Tree has a command wrapper for *drag-over* events. This command enables auto scrolling and position extraction during the *drag-over*. If **dropovercmd** is not empty, the command is called with the following aguments:
- the pathname of the tree,
- the pathname of the drag source,
- a list describing where the drop can occur, whose elements are:
- the current operation,
- the data type,
- the data.

- the string *widget* if **dropovertype** option contains *w*, else empty string.
- the targeted node if drag icon points a node and **dropovertype** option contains *n*, else empty string.
- a list containing a node and the position within the children of the node where drag icon points to if **dropovertype** option contains *p*, else empty string.
- optionally, the preferred method if drop can occur both inside a node and between two nodes. The value is *position* or *node*.

The command must return a list with two elements:
- the drop status, conforming to those described in **dropovercmd** option of **DropSite::register**,
- the choosen method: *widget*, *node* or *position*.

-dropovermode
Specifies the type of *drop-over* interaction. Must be a combination of **w**, which specifies that drop can occurs everywhere on widget, **p**, which specifies that drop can occurs between two nodes, and **n**, which specifies that drop occurs inside nodes.

-droptypes
Specifies a list of accepted dropped object/operation. See option **droptypes** of **DropSite::register**. for more infromation. Default is *TREE_NODE* with operations **copy** and **move**.

-height
Specifies the desired height for the tree in units of **deltay** pixels.

-linesfill
Specifies a foreground color for the lines between nodes.

-linestipple
Specifies a stipple bitmap for the lines between nodes.

-opencmd
Specifies a command to be called when the user opens a node. The name of the opened node is appended to the command.

-padx
Specifies distance between image or window and text of the nodes.

-redraw
Specifies wether or not the tree should be redrawn when entering idle. Set it to false if you call **update** while modifying the tree.

-selectcommand
Specifies a command to be called when the selection is changed. The path of the tree widget and the selected nodes are appended to the command.

-selectfill
If true, the selection box will be drawn across the entire tree from left-to-right instead of just around the item text.

-showlines
Specifies whether or not lines should be drawn between nodes.

-width
Specifies the desired width for the tree in units of 8 pixels.

NODE NAMES

Certain special characters in node names are automatically substituted by the tree during operation. These characters are **& | ^!**. They are all substituted with a _ character. This is only to avoid errors because the characters are special to the tree widget.

WIDGET COMMAND

pathName* bindArea *event script
This command associates a command to execute whenever the event sequence given by *event* occurs anywhere within the Tree area.

pathName* bindImage *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the image of a node. The node idenfier on which the event occurs is appended to the command and may be used to manipulate the tree (e.g. don't use **%W**).

If **-selectfill** is given, an eventual binding of the background box by **bindText** is overwritten.

pathName* bindText *event script
This command associates a command to execute whenever the event sequence given by *event* occurs on the label of a node. The node idenfier on which the event occurs is appended to the command and may be used to manipulate the tree (e.g. don't use **%W**).

If **-selectfill** is given, an eventual binding of the background box by **bindImage** is overwritten.

pathName* cget *option
Returns the current value of the configuration option given by *option*. *Option* may have any of the values accepted by the creation command.

***pathName* closetree *node* ?*recurse*?**
This command close all the subtree given by *node*. Recurse through the tree starting at *node* and set **open** option to 0 depending on *recurse*. Default value of *recurse* is *true*.

***pathName* configure ?*option*? ?*value option value ...*?**
Query or modify the configuration options of the widget. If no *option* is specified, returns a list describing all of the available options for *pathName*. If *option* is specified with no *value*, then the command returns a list describing the one named *option* (this list will be identical to the corresponding sublist of the value returned if no *option* is specified). If one or more *option-value* pairs are specified, then the command modifies the given widget option(s) to have the given value(s); in this case the command returns an empty string. *Option* may have any of the values accepted by the creation command. Read-only options are not be modified.

***pathName* delete ?*arg...*?**
Deletes all nodes (and children of them) in *arg*. *arg* can be a list of nodes or a list of list of nodes. To delete all the tree, do *$pathName delete [$pathName nodes root]*.

***pathName* edit *node text* ?*verifycmd*? ?*clickres*? ?*select*?**

Provides a way for the user to edit in place the label of a node. This is possible only if *node* is visible (all its parents are open). The command takes the initial text as argument and does not modify the label of the edited node, but returns an empty string if edition is canceled, or the typed text if edition is accepted. When editing, the user can cancel by pressing Escape, or accept by pressing Return. *clickres* specifies what to do if the user click outside the editable area. If *clickres* is 0 (the default), the edition is canceled. If *clickres* is 1, the edition is accepted. In all other case, the edition continues. If edition is accepted and *modifycmd* is not empty, then it is called with the new text as argument and must return 1 to accept the new text, 0 to refuse it and continue edition. *select* specifies wether or not the initial text should be selected. Default is 1.

pathName* exists *node

Returns whether or not *node* exists in the tree.

***pathName* find *findinfo* ?*confine*?**

Returns the node given by the position *findinfo*. *findinfo* can take the form of a pixel position *@x,y* or of the line number of a currently visible Tree node. The first line of the Tree has the value of zero.

If *confine* is non-empty, then confine *findinfo* to only match pixel positions for the area consumed by Tree labels, not just anywhere on their lines. (*confine* has no effect if *findinfo* is a line number.)

pathName* index *node

Returns the position of *node* in its parent.

***pathName* insert *index parent node* ?*option value...*?**

Inserts a new node identified by *node* in the children list of *parent* at position *index*.

Any instance of *#auto* within the node name will be replaced by the number of the item in the order of insertion. The non-printable characters \1 to \5 are reserved for internal use and should not be present in node names.

-anchor

Specifies the anchor of the image or window of the node. Defaults to w.

-data

User data associated to the node.

-deltax

Specifies the horizontal indentation of the node. If the value is -1, the node will be drawn with the deltax for the entire tree.

-drawcross

Specifies how the cross used to expand or collapse the children of a node should be drawn. Must be one of **auto**, **always** or **never**. If **auto**, the cross is drawn only if the node has children. If **always**, the cross is always drawn. If **never**, the cross is never drawn. To maintain compatibility with older versions of this widget, **allways** is a deprecated synonym to **always**.

-fill

Specifies the foreground color of the label of the node.

-font

Specifies a font for the label of the node.

-helpcmd
If specified, refers to a command to execute to get the help text to display. The command must return a string to display. If the command returns an empty string, no help is displayed. See also DynamicHelp.

-helptext
Text for dynamic help. See also DynamicHelp.

-helptype
Type of dynamic help. Use *balloon* or *variable*. See also DynamicHelp.

-helpvar
Variable to use when **helptype** option is *variable*. See also DynamicHelp.

-image
Specifies an image to display at the left of the label of the node. **window** option override **image**.

-open
Specifies wether or not the children of the node should be drawn.

-padx
Specifies the distance between image or window and the text of the node. If the value is -1, the node will be drawn with the padx for the entire tree.

-selectable
Specifies if the node can be selected or not.

-text
Specifies the label of the node.

-window
Specifies a pathname to display at the left of the label of the node. **window** option override **image**.

pathName* itemcget *node option
Returns the current value of a configuration option for the item. *Option* may have any of the values accepted by the item creation command.

***pathName* itemconfigure *node* ?*option*? ?*value option value ...*?**
This command is similar to the **configure** command, except that it applies to the options for an individual item, whereas **configure** applies to the options for the widget as a whole. **Options** may have any of the values accepted by the item creation widget command. If options are specified, options are modified as indicated in the command and the command returns an empty string. If no options are specified, returns a list describing the current options for the item. Read-only options are not be modified.

pathName* line *node
Returns the line number where *node* was drawn. If the node is not visible then return -1. The first line of the tree has the value of 0.

pathName* move *parent node index
Moves *node* to the children list of *parent* at position *index*. *parent* can not be a descendant of *node*.

***pathName* nodes *node* ?*first*? ?*last*?**
Returns parts of the children of *node*, following *first* and *last*. If *first* and *last* are omitted,

returns the list of all children. If *first* is specified and *last* omitted, returns the child at index *first*, or an empty string if *first* refers to a non-existent element. If *first* and *last* are specified, the command returns a list whose elements are all of the children between *first* and *last*, inclusive. Both *first* and *last* may have any of the standard forms for indices.

***pathName* opentree *node* ?*recurse*?**
This command open all the subtree given by *node*. Recurse through the tree starting at *node* and set **open** option to 1 depending on value of *recurse*. Default value of *recurse* is *true*.

pathName* parent *node
Returns the parent of *node*.

pathName* reorder *node neworder
Modifies the order of children of *node* given by *neworder*. Children of *node* that do not appear in *neworder* are no moved.

pathName* see *node
Arrange the scrolling area to make *node* visible.

***pathName* selection *cmd* ?*arg...*?**
Modifies the list of selected nodes following *cmd*:

add
Adds all nodes in *arg* to the selection.

clear
Removes all nodes from the selection.

get
Returns a list containing the indices of current selected nodes.

includes
Tests if the specified node is selected. Returns true if the answer is yes, and false else.

range
Sets the selection to all nodes between the two specified ones.

remove
Removes all nodes in *arg* from the selection.

set
Sets the selection to all nodes in *arg*.

toggle
Toggles the selection status of all nodes in *arg*.

The subcommands **add**, **range**, and **set** silently ignore nodes which are declared unselectable. See the node option -selectable to influence this.

pathName* toggle *node
Toggle the open/close status of the given *node*.

pathName* visible *node
Returns whether or not *node* is visible (all its parents are open).

***pathName* xview ?*arg...*?**
Standard command to enable horizontal scrolling of *pathName*.

***pathName* yview ?*arg...*?**

Standard command to enable vertical scrolling of *pathName*.

BINDINGS

A **<<TreeSelect>>** virtual event is generated any time the selection in the tree changes. This is the default behavior of an item in the tree, but it can be overridden with the bindText or bindImage command. If the button 1 binding is overridden, this event may not be generated.

The tree has all the standard mouse wheel bindings when it has focus.

Widget

Keywords: Widget

Under construction ...

NAME

Widget - The Widget base class

COMMAND

Widget::addmap *class subclass subpath options*

Widget::bwinclude *class subclass subpath* ?*arg...*?

Widget::cget *path option*

Widget::configure *path options*

Widget::create *class path* ?*rename*?

Widget::declare *class optlist*

Widget::define *class filename* ?*class ...*?

Widget::destroy *path*

Widget::focusNext *w*

Widget::focusOK *w*

Widget::focusPrev *w*

Widget::generate-doc *dir widgetlist*

Widget::generate-widget-doc *class iscmd file*

Widget::getoption *path option*

Widget::getVariable *path varName?myVarName?*

Widget::hasChanged *path option pvalue*

Widget::init *class path options*

Widget::setoption *path option value*

Widget::subcget *path subwidget*

Widget::syncoptions *class subclass subpath options*

Widget::tkinclude *class tkwidget subpath* ?*arg...*?

DESCRIPTION

The **Widget** namespace handle data associated to all BWidget and provide commands to easily define BWidget.
For commands can be used to define a BWidget: **tkinclude**, **bwinclude**, **declare**, **addmap** and **syncoptions**. Here is the definition of ComboBox widget:

```
namespace eval ComboBox {
    # We're using ArrowButton, Entry and LabelFrame
    ArrowButton::use
    Entry::use
    LabelFrame::use

    # Include resources of LabelFrame
    Widget::bwinclude ComboBox LabelFrame .labf \
        rename     {-text -label} \
        remove     {-focus} \
        prefix     {label -justify -width -anchor -height -font} \
        initialize {-relief sunken -borderwidth 2}

    # Include resources of Entry
    Widget::bwinclude ComboBox Entry .e \
        remove {-relief -bd -borderwidth -bg -fg} \
        rename {-foreground -entryfg -background -entrybg}

    # Declare new resources
    Widget::declare ComboBox {
        {-height      TkResource 0  0 listbox}
        {-values      String     "" 0}
        {-modifycmd   String     "" 0}
        {-postcommand String     "" 0}
    }

    # Map resources to subwidget
    Widget::addmap ComboBox "" :cmd {-background {}}
    Widget::addmap ComboBox ArrowButton .a \
        {-foreground {} -background {} -disabledforeground {} -state {}}

    # Synchronize subwidget options
    Widget::syncoptions ComboBox Entry .e {-text {}}
```

```
    Widget::syncoptions ComboBox LabelFrame .labf {-label -text -underline {}}

    proc use {} {}
}
```

COMMAND

Widget::addmap *class subclass subpath options*

This command map some resources to subwidget. Mapped resources automatically configure subwidget when widget is configured.
- *class* is the class of the new BWidget
- *subclass* is the class the subwidget (BWidget class, e.g Entry, or empty for Tk widget)
- *subpath* is the path of the subwidget
- *options* is the list *{option realres ...}* of options to map to subwidget

Widget::bwinclude *class subclass subpath ?arg...?*

This command includes into a new BWidget the resources of another BWidget. Arguments are:
- *class* class of the new widget
- *subclass* class name of the BWidget to be included
- *subpath* path of the widget to configure when BWidget is configured
- *options* is:
  - *include {option option ...}* list of options to include (all if not defined)
  - *remove {option option ...}* list of options to remove
  - *rename {option name option name ...}* list of options to rename
  - *prefix {prefix option option ...}* pefix all *option* by *prefix*
  - *initialize {option value option value ...}* default value of options
  - *readonly {option value option value ...}* new readonly flag

Widget::cget *path option*

Returns the value of *option* of BWidget *path*. **cget** tests the option existence and takes care of synchronization with subwidget. Typically called by the BWidget **cget** command.

Widget::configure *path options*

Description text

Widget::create *class path ?rename?*

The standard method for creating a BWidget. The real widget path is renamed to $path:cmd, and a new proc is created to replace the path which points to the BWidget's commands.

If *rename* is false, the path will not be renamed, but the proc will still be created. This is useful when inheriting another BWidget who will already have renamed the widget.

The command returns the widget path. This command is usually the last command executed in the ::create command for the widget.

Widget::declare *class optlist*

This command declare new resources for a BWidget.
- *class* is class of the new widget
- *options* is the list describing new options. Each new option is a list **{option type value ro**

?args?} where:
- *option* is the name of the option
- *type* is the type of the option
- *value* is the default value of the option
- *ro* is the readonly flag of the option
- *args* depends on type

type can be:

TkResource
value of *option* denotes a resource of a Tk widget. *args* must be *class* or *{class realoption}*. *class* is the creation command of the Tk widget, e.g. **entry**. The second form must be used if *option* has not the same name in Tk widget, but *realoption*. If *value* is empty, it is initialized to the default value of the Tk widget.

BwResource
value of *option* denotes a resource of a BWidget. *args* must be *class* or *{class realoption}*. *class* is the name of the namespace of the BWidget, e.g. **LabelFrame**. The second form must be used if *option* has not the same name in BWidget, but *realoption*. If *value* is empty, it is initialized to the default value of the BWidget.

Int
value of *option* is an integer. *args* can be *{?min? ?max?}* to force it to be in a range. The test is *[expr $option > $min] && [expr $option < $max]* so if args is *{0 10}*, value must be beetween 0 and 10 exclude, if *args* is *{=0 =10}*, value must be beetween 0 and 10 include.

Boolean
value of *option* is a boolean. True values can be **1**, **true** or **yes**. False values can be **0**, **false** or **no**. **Widget::cget** always return 0 or 1.

Enum
value of *option* is a element of a enumeration. *args* must be the list of enumeration, e.g. *{top left bottom right}*.

Flag
value of *option* is a combination of a set of chars. *args* must be a string defining the set.

String
value of *option* is any uncontrolled string.

Synonym
option is a synonym of option *args*. *value* has no effect here.

Widget::define *class filename ?class ...?*

This command is used to define a new BWidget class. It is usually the first command executed in a new widget definition.
- *class* is the name of the new widget class.
- *filename* is the name of the file (without extension) in the BWidget distribution that defines this class.

Each class defined after the filename is a class that this widget depends on. The ::use command will be called for each of these classes after the new widget has been defined.

This command does several things to setup the new class. First, it creates an alias in the global namespace for the name of the class that points to the class's ::create subcommand. Second, it defines a ::use subcommand for the class which other classes can use to load this class on the fly. Lastly, it creates a default binding to the <Destroy> event for the class that calls Widget::destroy on the path. This is the default setup for almost all widgets in the BWidget package.

Widget::destroy *path*
Description text

Widget::focusNext *w*
Description text

Widget::focusOK *w*
Description text

Widget::focusPrev *w*
Description text

Widget::generate-doc *dir widgetlist*
Description text

Widget::generate-widget-doc *class iscmd file*
Description text

Widget::getoption *path option*
Returns the value of *option* of BWidget *path*. This command does not test option existence, does not handle synonym and does not take care of synchronization with subwidget.

Widget::getVariable *path varName?myVarName?*
Make the variable *varName* relational to *path* accessible in the current procedure. The variable will be created in the widget namespace for *path* and can be used for storing widget-specific information. When *path* is destroyed, any variable accessed in this manner will be destroyed with it.

If *myVarName* is specified, the variable will be accessible in the current procedure as that name.

Widget::hasChanged *path option pvalue*
Description text

Widget::init *class path options*
Description text

Widget::setoption *path option value*
Set the value of *option* of BWidget *path* without option test, subwidget mapping, synonym handling and does not set the modification flag.

Widget::subcget *path subwidget*
Returns the list of all option/value of BWidget *path* that are mapped to *subwidget*.

Widget::syncoptions *class subclass subpath options*
This command synchronize options value of a subwidget. Used when an option of a subwidget is modified out of the BWidget **configure** command.

- *class* is the class of the new BWidget
- *subclass* is the class the subwidget (BWidget class, e.g Entry, or empty for Tk widget)
- *subpath* is the path of the subwidget
- *options* is the list *{option realres ...}* of options to synchronize with subwidget

Widget::tkinclude *class tkwidget subpath* ?*arg...*?

This command includes into a new BWidget the resources of a Tk widget. Arguments are:
- *class* class of the new widget
- *tkwidger* command name of the Tk widget to be included
- *subpath* path of the widget to configure when BWidget is configured
- *options* is:
  - *include {option option ...}* list of options to include (all if not defined)
  - *remove {option option ...}* list of options to remove
  - *rename {option name option name ...}* list of options to rename
  - *prefix {prefix option option ...}* pefix all *option* by *prefix*
  - *initialize {option value option value ...}* default value of options
  - *readonly {option value option value ...}* new readonly flag

contents

Keywords: contents

Simple Widgets

Label	Label widget with state option, dynamic help and drag and drop facilities
Entry	Entry widget with state option, dynamic help and drag and drop facilities
Button	Button widget with enhanced options
ArrowButton	Button widget with an arrow shape.
ProgressBar	Progress indicator widget
ScrollView	Display the visible area of a scrolled window
Separator	3D separator widget

Manager Widgets

MainFrame	Manage toplevel with menu, toolbar and statusbar
LabelFrame	Frame with a Label
TitleFrame	Frame with a title (consider Tk 8.4+ labelframe instead)
PanelFrame	Frame with a boxed title area

ScrolledWindow	Generic scrolled widget
ScrollableFrame	Scrollable frame containing widget
PanedWindow	Tiled layout manager widget (consider Tk 8.4+ panedwindow instead)
ButtonBox	Set of buttons with horizontal or vertical layout
PagesManager	Pages manager widget
NoteBook	Notebook manager widget
Dialog	Dialog abstraction with custom buttons
StatusBar	Status bar widget with resize control

Composite Widgets

LabelEntry	LabelFrame containing an Entry widget.
ComboBox	ComboBox widget
SpinBox	SpinBox widget (consider Tk 8.4+ spinbox instead)
Tree	Tree widget
ListBox	ListBox widget
MessageDlg	Message dialog box
ProgressDlg	Progress indicator dialog box
PasswdDlg	Login/Password dialog box
SelectFont	Font selection widget
SelectColor	Color selection widget

Commands Classes

Widget	The Widget base class
DynamicHelp	Provide help to Tk widget or BWidget
DragSite	Commands set for Drag facilities
DropSite	Commands set for Drop facilities
BWidget	Description text

index

Keywords: index

navtree

Keywords: navtree

Brief description

Simple Widgets

Label

Entry

Button

ArrowButton

ProgressBar

ScrollView

Separator

Manager Widgets

MainFrame

LabelFrame

TitleFrame

PanelFrame

ScrolledWindow

ScrollableFrame

PanedWindow

ButtonBox

PagesManager

NoteBook

Dialog

StatusBar

Composite Widgets

LabelEntry

ComboBox

SpinBox

Tree

ListBox

MessageDlg

ProgressDlg

PasswdDlg

SelectFont

SelectColor

Commands Classes

Widget

DynamicHelp

DragSite

DropSite

BWidget

options

Keywords: options

NAME

options - Standard options supported by widgets

DESCRIPTION

-activebackground, activeBackground, Foreground

-activeborderwidth, activeBorderWidth, BorderWidth

-activeforeground, activeForeground, Background

-anchor, anchor, Anchor

-background or -bg, background, Background

-bitmap, bitmap, Bitmap

-borderwidth or -bd, borderWidth, BorderWidth

-cursor, cursor, Cursor

-disabledbackground, disabledBackground, DisabledBackground

-disabledforeground, disabledForeground, DisabledForeground

-exportselection, exportSelection, ExportSelection

-font, font, Font

-foreground or -fg, foreground, Foreground

-highlightbackground, highlightBackground, HighlightBackground

-highlightcolor, highlightColor, HighlightColor

-highlightthickness, highlightThickness, HighlightThickness

-image, image, Image

-insertbackground, insertBackground, Foreground

-insertborderwidth, insertBorderWidth, BorderWidth

-insertofftime, insertOffTime, OffTime

-insertontime, insertOnTime, OnTime

-insertwidth, insertWidth, InsertWidth

-jump, jump, Jump

-justify, justify, Justify

-orient, orient, Orient

-padx, padX, Pad

-pady, padY, Pad

-relief, relief, Relief

-repeatdelay, repeatDelay, RepeatDelay

-repeatinterval, repeatInterval, RepeatInterval

-selectbackground, selectBackground, Foreground

-selectborderwidth, selectBorderWidth, BorderWidth

-selectforeground, selectForeground, Background

-setgrid, setGrid, SetGrid

-takefocus, takeFocus, TakeFocus

-text, text, Text

-textvariable, textVariable, Variable

-troughcolor, troughColor, Background

-underline, underline, Underline

-wraplength, wrapLength, WrapLength

-xscrollcommand, xScrollCommand, ScrollCommand

-yscrollcommand, yScrollCommand, ScrollCommand

KEYWORDS

NAME

options - Standard options supported by widgets

DESCRIPTION

This manual entry describes the common configuration options supported by widgets in the Tk toolkit. Every widget does not necessarily support every option (see the manual entries for individual widgets for a list of the standard options supported by that widget), but if a widget does support an option with one of the names listed below, then the option has exactly the effect described below.

In the descriptions below, ``Command-Line Name'' refers to the switch used in class commands and **configure** widget commands to set this value. For example, if an option's command-line switch is **-foreground** and there exists a widget **.a.b.c**, then the command

```
.a.b.c  configure  -foreground black
```

may be used to specify the value

black

for the option in the the widget

.a.b.c

. Command-line switches may be abbreviated, as long as the abbreviation is unambiguous. ``Database Name'' refers to the option's name in the option database (e.g. in .Xdefaults files). ``Database Class'' refers to the option's class value in the option database.

Database Class: Foreground

Specifies background color to use when drawing active elements. An element (a widget or portion of a widget) is active if the mouse cursor is positioned over the element and pressing a mouse button will cause some action to occur. If strict Motif compliance has been requested by setting the **tk_strictMotif** variable, this option will normally be ignored; the normal background color will be used instead. For some elements on Windows and Macintosh systems, the active color will only be used while mouse button 1 is pressed over the element.

Command-Line Name: -activeborderwidth Database Name: activeBorderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border drawn around active elements. See above for definition of active elements. The value may have any of the forms acceptable to **Tk_GetPixels**. This option is typically only available in widgets displaying more

than one element at a time (e.g. menus but not buttons).

Command-Line Name: -activeforeground Database Name: activeForeground Database Class: Background

Specifies foreground color to use when drawing active elements. See above for definition of active elements.

Command-Line Name: -anchor Database Name: anchor Database Class: Anchor

Specifies how the information in a widget (e.g. text or a bitmap) is to be displayed in the widget. Must be one of the values **n**, **ne**, **e**, **se**, **s**, **sw**, **w**, **nw**, or **center**. For example, **nw** means display the information such that its top-left corner is at the top-left corner of the widget.

Command-Line Name: -background or -bg Database Name: background Database Class: Background

Specifies the normal background color to use when displaying the widget.

Command-Line Name: -bitmap Database Name: bitmap Database Class: Bitmap

Specifies a bitmap to display in the widget, in any of the forms acceptable to **Tk_GetBitmap**. The exact way in which the bitmap is displayed may be affected by other options such as **anchor** or **justify**. Typically, if this option is specified then it overrides other options that specify a textual value to display in the widget; the **bitmap** option may be reset to an empty string to re-enable a text display. In widgets that support both **bitmap** and **image** options, **image** will usually override **bitmap**.

Command-Line Name: -borderwidth or -bd Database Name: borderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border to draw around the outside of the widget (if such a border is being drawn; the **relief** option typically determines this). The value may also be used when drawing 3-D effects in the interior of the widget. The value may have any of the forms acceptable to **Tk_GetPixels**.

Command-Line Name: -cursor Database Name: cursor Database Class: Cursor

Specifies the mouse cursor to be used for the widget. The value may have any of the forms acceptable to **Tk_GetCursor**.

Command-Line Name: **-disabledforeground**

Command-Line Name: -disabledbackground Database Name: disabledBackground Database Class: DisabledBackground Specifies background color to use when drawing a disabled

element. If the option is specified as an empty string (which is typically the case on monochrome displays), disabled elements are drawn with the normal background color but they are dimmed by drawing them with a stippled fill pattern. Database Name: disabledForeground Database Class: DisabledForeground

Specifies foreground color to use when drawing a disabled element. If the option is specified as an empty string (which is typically the case on monochrome displays), disabled elements are drawn with the normal foreground color but they are dimmed by drawing them with a stippled fill pattern.

Command-Line Name: -exportselection Database Name: exportSelection Database Class: ExportSelection

Specifies whether or not a selection in the widget should also be the X selection. The value may have any of the forms accepted by **Tcl_GetBoolean**, such as **true**, **false**, **0**, **1**, **yes**, or **no**. If the selection is exported, then selecting in the widget deselects the current X selection, selecting outside the widget deselects any widget selection, and the widget will respond to selection retrieval requests when it has a selection. The default is usually for widgets to export selections.

Command-Line Name: -font Database Name: font Database Class: Font

Specifies the font to use when drawing text inside the widget.

Command-Line Name: -foreground or -fg Database Name: foreground Database Class: Foreground

Specifies the normal foreground color to use when displaying the widget.

Command-Line Name: -highlightbackground Database Name: highlightBackground Database Class: HighlightBackground

Specifies the color to display in the traversal highlight region when the widget does not have the input focus.

Command-Line Name: -highlightcolor Database Name: highlightColor Database Class: HighlightColor

Specifies the color to use for the traversal highlight rectangle that is drawn around the widget when it has the input focus.

Command-Line Name: -highlightthickness Database Name: highlightThickness Database Class: HighlightThickness

Specifies a non-negative value indicating the width of the highlight rectangle to draw around the outside of the widget when it has the input focus. The value may have any of the forms

acceptable to **Tk_GetPixels**. If the value is zero, no focus highlight is drawn around the widget.

Command-Line Name: -image Database Name: image Database Class: Image

Specifies an image to display in the widget, which must have been created with the **image create** command. Typically, if the **image** option is specified then it overrides other options that specify a bitmap or textual value to display in the widget; the **image** option may be reset to an empty string to re-enable a bitmap or text display.

Command-Line Name: -insertbackground Database Name: insertBackground Database Class: Foreground

Specifies the color to use as background in the area covered by the insertion cursor. This color will normally override either the normal background for the widget (or the selection background if the insertion cursor happens to fall in the selection).

Command-Line Name: -insertborderwidth Database Name: insertBorderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border to draw around the insertion cursor. The value may have any of the forms acceptable to **Tk_GetPixels**.

Command-Line Name: -insertofftime Database Name: insertOffTime Database Class: OffTime

Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain ``off'' in each blink cycle. If this option is zero then the cursor doesn't blink: it is on all the time.

Command-Line Name: -insertontime Database Name: insertOnTime Database Class: OnTime

Specifies a non-negative integer value indicating the number of milliseconds the insertion cursor should remain ``on'' in each blink cycle.

Command-Line Name: -insertwidth Database Name: insertWidth Database Class: InsertWidth

Specifies a value indicating the total width of the insertion cursor. The value may have any of the forms acceptable to **Tk_GetPixels**. If a border has been specified for the insertion cursor (using the **insertBorderWidth** option), the border will be drawn inside the width specified by the **insertWidth** option.

Command-Line Name: -jump Database Name: jump Database Class: Jump

For widgets with a slider that can be dragged to adjust a value, such as scrollbars, this option determines when notifications are made about changes in the value. The option's value must be a boolean of the form accepted by **Tcl_GetBoolean**. If the value is false, updates are

made continuously as the slider is dragged. If the value is true, updates are delayed until the mouse button is released to end the drag; at that point a single notification is made (the value ``jumps'' rather than changing smoothly).

Command-Line Name: -justify Database Name: justify Database Class: Justify

When there are multiple lines of text displayed in a widget, this option determines how the lines line up with each other. Must be one of **left**, **center**, or **right**. **Left** means that the lines' left edges all line up, **center** means that the lines' centers are aligned, and **right** means that the lines' right edges line up.

Command-Line Name: -orient Database Name: orient Database Class: Orient

For widgets that can lay themselves out with either a horizontal or vertical orientation, such as scrollbars, this option specifies which orientation should be used. Must be either **horizontal** or **vertical** or an abbreviation of one of these.

Command-Line Name: -padx Database Name: padX Database Class: Pad

Specifies a non-negative value indicating how much extra space to request for the widget in the X-direction. The value may have any of the forms acceptable to **Tk_GetPixels**. When computing how large a window it needs, the widget will add this amount to the width it would normally need (as determined by the width of the things displayed in the widget); if the geometry manager can satisfy this request, the widget will end up with extra internal space to the left and/or right of what it displays inside. Most widgets only use this option for padding text: if they are displaying a bitmap or image, then they usually ignore padding options.

Command-Line Name: -pady Database Name: padY Database Class: Pad

Specifies a non-negative value indicating how much extra space to request for the widget in the Y-direction. The value may have any of the forms acceptable to **Tk_GetPixels**. When computing how large a window it needs, the widget will add this amount to the height it would normally need (as determined by the height of the things displayed in the widget); if the geometry manager can satisfy this request, the widget will end up with extra internal space above and/or below what it displays inside. Most widgets only use this option for padding text: if they are displaying a bitmap or image, then they usually ignore padding options.

Command-Line Name: -relief Database Name: relief Database Class: Relief

Specifies the 3-D effect desired for the widget. Acceptable values are **raised**, **sunken**, **flat**, **ridge**, **solid**, and **groove**. The value indicates how the interior of the widget should appear relative to its exterior; for example, **raised** means the interior of the widget should appear to protrude from the screen, relative to the exterior of the widget.

Command-Line Name: -repeatdelay Database Name: repeatDelay Database Class:

RepeatDelay

Specifies the number of milliseconds a button or key must be held down before it begins to auto-repeat. Used, for example, on the up- and down-arrows in scrollbars.

Command-Line Name: -repeatinterval Database Name: repeatInterval Database Class: RepeatInterval

Used in conjunction with **repeatDelay**: once auto-repeat begins, this option determines the number of milliseconds between auto-repeats.

Command-Line Name: -selectbackground Database Name: selectBackground Database Class: Foreground

Specifies the background color to use when displaying selected items.

Command-Line Name: -selectborderwidth Database Name: selectBorderWidth Database Class: BorderWidth

Specifies a non-negative value indicating the width of the 3-D border to draw around selected items. The value may have any of the forms acceptable to **Tk_GetPixels**.

Command-Line Name: -selectforeground Database Name: selectForeground Database Class: Background

Specifies the foreground color to use when displaying selected items.

Command-Line Name: -setgrid Database Name: setGrid Database Class: SetGrid

Specifies a boolean value that determines whether this widget controls the resizing grid for its top-level window. This option is typically used in text widgets, where the information in the widget has a natural size (the size of a character) and it makes sense for the window's dimensions to be integral numbers of these units. These natural window sizes form a grid. If the **setGrid** option is set to true then the widget will communicate with the window manager so that when the user interactively resizes the top-level window that contains the widget, the dimensions of the window will be displayed to the user in grid units and the window size will be constrained to integral numbers of grid units. See the section GRIDDED GEOMETRY MANAGEMENT in the **wm** manual entry for more details.

Command-Line Name: -takefocus Database Name: takeFocus Database Class: TakeFocus

Determines whether the window accepts the focus during keyboard traversal (e.g., Tab and Shift-Tab). Before setting the focus to a window, the traversal scripts consult the value of the **takeFocus** option. A value of **0** means that the window should be skipped entirely during keyboard traversal. **1** means that the window should receive the input focus as long as it is viewable (it and all of its ancestors are mapped). An empty value for the option means that

the traversal scripts make the decision about whether or not to focus on the window: the current algorithm is to skip the window if it is disabled, if it has no key bindings, or if it is not viewable. If the value has any other form, then the traversal scripts take the value, append the name of the window to it (with a separator space), and evaluate the resulting string as a Tcl script. The script must return **0**, **1**, or an empty string: a **0** or **1** value specifies whether the window will receive the input focus, and an empty string results in the default decision described above. Note: this interpretation of the option is defined entirely by the Tcl scripts that implement traversal: the widget implementations ignore the option entirely, so you can change its meaning if you redefine the keyboard traversal scripts.

Command-Line Name: -text Database Name: text Database Class: Text

Specifies a string to be displayed inside the widget. The way in which the string is displayed depends on the particular widget and may be determined by other options, such as **anchor** or **justify**.

Command-Line Name: -textvariable Database Name: textVariable Database Class: Variable

Specifies the name of a variable. The value of the variable is a text string to be displayed inside the widget; if the variable value changes then the widget will automatically update itself to reflect the new value. The way in which the string is displayed in the widget depends on the particular widget and may be determined by other options, such as **anchor** or **justify**.

Command-Line Name: -troughcolor Database Name: troughColor Database Class: Background

Specifies the color to use for the rectangular trough areas in widgets such as scrollbars and scales.

Command-Line Name: -underline Database Name: underline Database Class: Underline

Specifies the integer index of a character to underline in the widget. This option is used by the default bindings to implement keyboard traversal for menu buttons and menu entries. 0 corresponds to the first character of the text displayed in the widget, 1 to the next character, and so on.

Command-Line Name: -wraplength Database Name: wrapLength Database Class: WrapLength

For widgets that can perform word-wrapping, this option specifies the maximum line length. Lines that would exceed this length are wrapped onto the next line, so that no line is longer than the specified length. The value may be specified in any of the standard forms for screen distances. If this value is less than or equal to 0 then no wrapping is done: lines will break only at newline characters in the text.

Command-Line Name: -xscrollcommand Database Name: xScrollCommand Database Class: ScrollCommand

Specifies the prefix for a command used to communicate with horizontal scrollbars. When the view in the widget's window changes (or whenever anything else occurs that could change the display in a scrollbar, such as a change in the total size of the widget's contents), the widget will generate a Tcl command by concatenating the scroll command and two numbers. Each of the numbers is a fraction between 0 and 1, which indicates a position in the document. 0 indicates the beginning of the document, 1 indicates the end, .333 indicates a position one third the way through the document, and so on. The first fraction indicates the first information in the document that is visible in the window, and the second fraction indicates the information just after the last portion that is visible. The command is then passed to the Tcl interpreter for execution. Typically the **xScrollCommand** option consists of the path name of a scrollbar widget followed by ``set'', e.g. ``.x.scrollbar set'': this will cause the scrollbar to be updated whenever the view in the window changes. If this option is not specified, then no command will be executed.

Command-Line Name: -yscrollcommand Database Name: yScrollCommand Database Class: ScrollCommand Specifies the prefix for a command used to communicate with vertical scrollbars. This option is treated in the same way as the xScrollCommand option, except that it is used for vertical scrollbars and is provided by widgets that support vertical scrolling. See the description of xScrollCommand for details on how this option is used.

KEYWORDS

class

,

name

,

standard option

,

switch

```
Copyright © 1990-1994 The Regents of the University of California.
Copyright © 1994-1996 Sun Microsystems, Inc.
Copyright © 1995-1997 Roger E. Critchlow Jr.
```

License

These pages were converted from the original manual pages to Markdown (TIP 700 format). The text is unchanged; only markup without a place in the format (links without a target, layout images) may be lost. For parts under the Mozilla Public License 1.1 (tDOM) this is the description of modifications required by section 3.3.

This collection is not an official publication of the projects. Each part is under the terms of its original, which follow. The copyright notices of the original pages stay with each page.

BWidget ToolKit

Copyright (c) 1998-1999 UNIFIX.

Copyright (c) 2001-2002 ActiveState Corp.

The following terms apply to all files associated with the software
unless explicitly disclaimed in individual files.
The authors hereby grant permission to use, copy, modify, distribute,
and license this software and its documentation for any purpose, provided
that existing copyright notices are retained in all copies and that this
notice is included verbatim in any distributions. No written agreement,
license, or royalty fee is required for any of the authorized uses.
Modifications to this software may be copyrighted by their authors
and need not follow the licensing terms described here, provided that
the new terms are clearly indicated on the first page of each file where
they apply.
IN NO EVENT SHALL THE AUTHORS OR DISTRIBUTORS BE LIABLE TO ANY PARTY
FOR DIRECT, INDIRECT, SPECIAL, INCIDENTAL, OR CONSEQUENTIAL DAMAGES
ARISING OUT OF THE USE OF THIS SOFTWARE, ITS DOCUMENTATION, OR ANY
DERIVATIVES THEREOF, EVEN IF THE AUTHORS HAVE BEEN ADVISED OF THE
POSSIBILITY OF SUCH DAMAGE.
THE AUTHORS AND DISTRIBUTORS SPECIFICALLY DISCLAIM ANY WARRANTIES,
INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE, AND NON-INFRINGEMENT. THIS SOFTWARE
IS PROVIDED ON AN "AS IS" BASIS, AND THE AUTHORS AND DISTRIBUTORS HAVE
NO OBLIGATION TO PROVIDE MAINTENANCE, SUPPORT, UPDATES, ENHANCEMENTS, OR
MODIFICATIONS.
GOVERNMENT USE: If you are acquiring this software on behalf of the
U.S. government, the Government shall have only "Restricted Rights"
in the software and related documentation as defined in the Federal
Acquisition Regulations (FARs) in Clause 52.227.19 (c) (2). If you
are acquiring the software on behalf of the Department of Defense, the
software shall be classified as "Commercial Computer Software" and the
Government shall have only "Restricted Rights" as defined in Clause
252.227-7013 (c) (1) of DFARs. Notwithstanding the foregoing, the
authors grant the U.S. Government and others acting in its behalf
permission to use and distribute the software in accordance with the
terms specified in this license.

Index

-

-activator 60
-anchor 20, 90
-arcradius 55
-areaheight 71
-areawidth 71
-armcommand 4, 9
-arrowbd 4
-arrowrelief 4
-aspect 52
-auto 73
-autocomplete 16
-autofocus 42
-autopost 16
-background 12
-baseline 83
-bitmap 20
-borderwidth 28
-buttons 52
-buttonwidth 52
-bwlistbox 16
-cancel 20
-clean 4
-closecmd 86
-color 74
-command 4, 9, 27, 30, 64, 68, 76
-constrainedheight 71
-constrainedwidth 71
-createcmd 56
-crossclosebitmap 86
-crosscloseimage 86
-crossfill 86
-crossopenbitmap 86
-crossopenimage 86
-data 45, 90
-default 9, 12, 20
-delay 28
-deltax 42, 86
-deltay 42, 86
-dir 5
-disarmcommand 5, 10
-dragenabled 30, 35, 42, 86
-dragendcmd 22, 30, 35, 42, 87

-dragevent 22, 30, 35, 42, 87
-draginitcmd 22, 30, 35, 42, 87
-dragtype 31, 35, 43, 87
-drawcross 90
-dropcmd 24, 31, 35, 43, 87
-dropenabled 31, 35, 43, 87
-dropovercmd 24, 31, 35, 43, 87
-dropovermode 44, 88
-droptypes 25, 32, 36, 44, 88
-editable 32
-expand 16, 62
-fill 46, 62, 69, 90
-focus 36
-font 46, 90
-foreground 69
-geometry 20
-height 5, 10, 16, 36, 44, 48, 55, 58, 62, 66, 68, 69, 71, 81, 88
-helpcmd 91
-helptext 5, 10, 32, 36, 91
-helptype 5, 10, 32, 36, 91
-helpvar 5, 10, 32, 36, 91
-homogeneous 13, 55
-icon 52
-image 20, 46, 56, 91
-indent 46
-index 27
-initialcolor 76
-ipad 62, 73, 82, 83
-ipadx 5
-ipady 5
-item 27
-justify 52
-leavecmd 56
-linesfill 88
-linestipple 88
-managed 73
-maximum 66
-menu 48
-message 52
-minsize 60
-modal 20
-modifycmd 16, 79
-multicolumn 44
-name 10, 36
-nosizes 76
-open 91

-opencmd 88
-orient 13
-pad 60, 62, 82
-padx 13, 44, 88
-pady 13
-panelbackground 62
-panelforeground 62
-parent 20, 74, 76
-place 20
-placement 75
-postcommand 16
-raisecmd 56
-range 80
-redraw 44, 88
-relief 10, 77
-sampletext 76
-scrollbar 73
-selectable 91
-selectcommand 88
-selectfill 44, 88
-selectmode 44
-separator 20, 49, 82
-show 32
-showlines 88
-showresize 81
-side 20, 40, 55, 60, 62, 83
-sides 73
-size 73
-spacing 13
-state 5, 10, 13, 28, 32, 36, 56
-sticky 82
-stop 68
-tabbevelsize 55
-tabpady 55
-text 27, 46, 56, 91
-textvariable 50
-title 20, 52, 75, 76
-topbackground 28
-transient 20
-type 5, 27, 53, 64, 66, 75, 76
-underline 10, 36
-values 16, 80
-variable 27, 66
-weight 60, 82
-weights 60
-width 5, 11, 32, 36, 44, 50, 53, 55, 58, 60, 62, 66, 68, 69, 71,

82, 89
-window 46, 69, 91
-xscrollincrement 71
-yscrollincrement 71

A

abortretryignore 53
add 13, 20, 47, 58, 60, 62, 82, 92
addindicator 50
addtoolbar 50
ArrowButton 3
at 8

B

below 8
bind 17, 38, 80
bindArea 89
bindImage 44, 89
bindtabs 55
bindText 45, 89
Boolean 96
Button 8
ButtonBox 11
BWidget 6
BWidget::assert 7
BWidget::badOptionString 7
BWidget::bindMouseWheel 7
BWidget::classes 7
BWidget::clonename 7
BWidget::focus 7
BWidget::get3dcolor 7
BWidget::getname 7
BWidget::grab 8
BWidget::inuse 8
BWidget::library 8
BWidget::lreorder 8
BWidget::parsetext 8
BWidget::place 8
BWidget::write 8
BWidget::wrongNumArgsString 8
BWidget::XLFDfont 7
BwResource 96

C

center 8
cget 6, 11, 13, 17, 21, 32, 36, 38, 40, 45, 50, 55, 58, 60, 62, 64, 66, 68, 69, 71, 73, 75, 77, 78, 80, 82, 83, 89
clear 47, 92

clearvalue 17
closetree 89
ComboBox 14
compute_size 56, 58
configure 6, 11, 13, 17, 21, 32, 36, 38, 40, 45, 50, 56, 58, 60, 62, 65, 66, 68, 70, 71, 73, 75, 77, 78, 80, 82, 83, 89
contents 98

D

default 12, 19
delete 13, 45, 56, 59, 62, 82, 89
Dialog 18
DragSite 22
DragSite::include 22
DragSite::register 22
DragSite::setdrag 23
draw 21
DropSite 23
DropSite::include 24
DropSite::register 24
DropSite::setcursor 26
DropSite::setdrop 26
DropSite::setoperation 26
DynamicHelp 26
DynamicHelp::add 27
DynamicHelp::configure 28
DynamicHelp::delete 28
DynamicHelp::include 28
DynamicHelp::register 28
DynamicHelp::sethelp 29

E

edit 45, 89
end 12, 19
enddialog 21
Entry 29
Enum 96
exists 45, 90

F

find 90
first 17, 80
Flag 96

G

get 17, 47, 92
getframe 21, 40, 50, 56, 59, 61, 62, 72, 73, 82, 84
getindicator 50
getlistbox 17

getmenu 50
getmenustate 50
gettoolbar 50
getvalue 17, 80

I

icursor 17
includes 92
index 13, 45, 56, 90, 100
insert 13, 45, 56, 90
Int 96
invoke 6, 11, 13, 21, 33
item 46
itemcget 14, 21, 46, 56, 91
itemconfigure 14, 21, 46, 56, 91
items 46, 62, 82

L

Label 33
LabelEntry 37
LabelFrame 39
LabelFrame::align 40
last 12, 17, 19, 80
line 91
ListBox 40

M

MainFrame 47
MessageDlg 51
move 46, 57, 91

N

navtree 100
next 17, 80
nodes 91
NoteBook 53

O

ok 64
opentree 92
options 101

P

page 57, 59
pages 57, 59
PagesManager 57
PanedWindow 59
PanelFrame 61
parent 92
PasswdDlg 63

post 17
previous 17, 80
ProgressBar 65
ProgressDlg 67

R

raise 57, 59
range 92
remove 47, 92
reorder 46, 92
right 8

S

ScrollableFrame 70
ScrolledWindow 72
ScrollView 68
see 46, 57, 72, 92
SelectColor 74
SelectColor::dialog 75
SelectColor::menu 75
SelectColor::setcolor 75
SelectFont 75
SelectFont::loadfont 77
selection 47, 92
Separator 77
set 47, 92
setfocus 14, 21, 37
setmenustate 51
setvalue 17, 80
setwidget 74
showstatusbar 51
showtoolbar 51
SpinBox 78
StatusBar 81
String 96
Synonym 96

T

TitleFrame 82
TkResource 96
toggle 92
Tree 84

U

unpost 17

V

visible 92

W

Widget 93
Widget::addmap 95
Widget::bwinclude 95
Widget::cget 95
Widget::configure 95
Widget::create 95
Widget::declare 95
Widget::define 96
Widget::destroy 97
Widget::focusNext 97
Widget::focusOK 97
Widget::focusPrev 97
Widget::getoption 97
Widget::getVariable 97
Widget::hasChanged 97
Widget::init 97
Widget::setoption 97
Widget::subcget 97
Widget::syncoptions 97
Widget::tkinclude 98
withdraw 21

X

xview 47, 72, 92

Y

yview 47, 72, 92